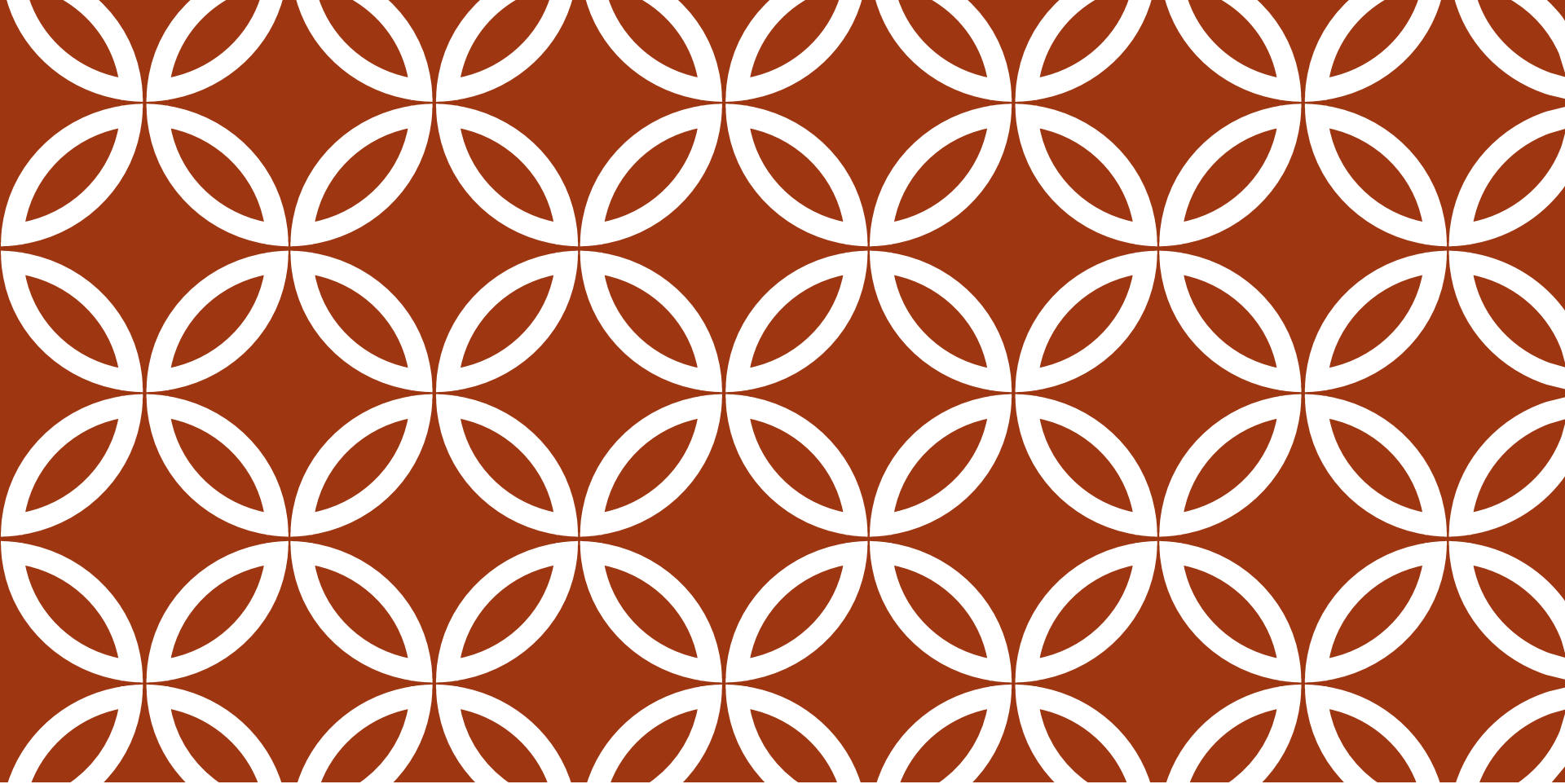




PLN

Ciência de Dados para Negócios
FATEC Votorantim

Prof. Luis Claudio

M. Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)



I – O QUE É PLN

Prof. Luis Claudio

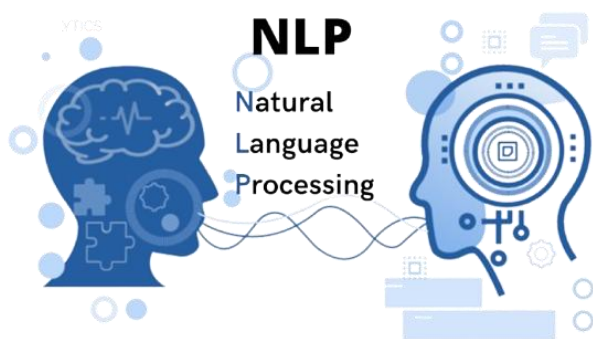
M.Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)

O QUE É PLN

O Processamento de Linguagem Natural (PLN) é o campo que estuda como “extrair informações não triviais e de alto valor” a partir da linguagem humana. O desafio é transformar textos oriundos da linguagem natural em algo **computacionalmente tratável**.

Por definição, a **linguagem natural** não possui estrutura formal explícita, ou seja, é não estruturada. Portanto, precisa ser transformada em **representações estruturadas** antes que técnicas analíticas possam ser aplicadas de forma sistemática.



PROPRIEDADE DA LINGUAGEM NATURAL

Ambiguidade (Polissemia)

Ambiguidade refere-se à possibilidade de uma **mesma expressão linguística admitir mais de uma interpretação legítima**. Ela pode ser **sintática**, quando diferentes estruturas gramaticais são possíveis, ou **semântica**, quando uma palavra possui múltiplos sentidos, exigindo mecanismos de desambiguação em sistemas de PLN.

- Já a palavra “**banco**” pode significar instituição financeira ou assento. Aqui não há mudança estrutural, mas sim múltiplos sentidos associados ao mesmo termo, caracterizando ambiguidade semântica (ou lexical).
- Na frase “**Vi o homem com o telescópio**”, há duas leituras estruturais plausíveis: ou eu utilizei o telescópio para ver o homem, ou o homem observado estava com o telescópio. É um caso de ambiguidade sintática.

PROPRIEDADE DA LINGUAGEM NATURAL

Variabilidade (Sinonímia)

A variabilidade diz respeito ao fato de que a **mesma ideia** pode ser expressa de muitas **maneiras diferentes**. Ao contrário da ambiguidade, aqui não existem vários sentidos possíveis. Existe um único sentido, mas com múltiplas formas de expressão. Isso obriga sistemas de PLN a reconhecer que textos diferentes podem estar dizendo essencialmente a mesma coisa.

- Por exemplo, “**rápido**”, “**veloz**” e “**ágil**” transmitem ideias próximas. Pior ainda ocorre com frases inteiras: “**o sistema não funcionou**”, “**o sistema caiu**”, “**o sistema travou**” ou “**deu pau**”. Todas descrevem o mesmo evento com escolhas linguísticas distintas. Essa multiplicidade de formas é natural na comunicação humana, mas aumenta demasiadamente a complexidade do processamento automático.

PROPRIEDADE DA LINGUAGEM NATURAL

Dependência de Contexto

A frase não tem vários sentidos disputando entre si, como acontece na ambiguidade. O problema é outro: a **informação linguística sozinha não basta**. Parte do significado vem da situação, do histórico da conversa ou do conhecimento compartilhado entre os interlocutores. Ou seja, o contexto não serve para escolher um sentido, mas para **completar** aquilo que não foi dito explicitamente.

- O significado de uma palavra depende das palavras ou da situação ao redor. Por exemplo: “banco” em “sentei no banco” versus “fui ao banco”. A dependência de contexto surge justamente porque existem fenômenos como polissemia e sinonímia
- Ou ainda, na frase “**Era um lugar absurdamente frio**”, também não há múltiplos sentidos disputando entre si. No entanto, sem contexto, não é possível determinar se a expressão é um exagero retórico (uma sala com ar-condicionado forte) ou a um ambiente extremo, como o Polo Norte.

PRINCIPAIS TAREFAS EM PLN

Podemos listar diversas **tarefas** (ou **problemas**) em PLN, isto é, o tipo de processamento que desejamos executar sobre texto ou fala.

- Na literatura, também é comum encontrar a expressão **problemas de PLN**, usada no sentido de formulação técnica de uma tarefa (por exemplo, um problema de classificação ou de tradução).

PRINCIPAIS TAREFAS EM PLN

Classificação Supervisionada

Classificação de textos consiste em **atribuir a um texto um rótulo dentre um conjunto finito e predefinido de classes**. Trata-se de um problema supervisionado no qual o modelo aprende, a partir de exemplos rotulados, uma função que mapeia representações vetoriais do texto para categorias específicas. A tarefa pode envolver classificação binária, multiclasse ou multilabel, dependendo do número e da natureza dos rótulos.

- Exemplo: dado um conjunto de **e-mails** rotulados como “**spam**” e “**não spam**”, o modelo aprende padrões linguísticos e estatísticos e passa a prever o rótulo de novos e-mails. Isso não é “agrupar por similaridade” (o que seria um aprendizado não supervisionado), pois a saída é um rótulo predefinido, não *clusters* descobertos. A classificação pode ser **multiclasse** inclusive (até dezenas...).

PRINCIPAIS TAREFAS EM PLN

Análise de Sentimento

É um **tipo de classificação de texto** que consiste em **inferir a polaridade ou atitude em um texto** (por exemplo, positivo, negativo ou neutro). Na prática, é formulada como um problema de **classificação supervisionada**, no qual o modelo aprende a mapear representações textuais para classes de polaridade a partir de exemplos previamente rotulados.

Existem abordagens baseadas em léxicos, nas quais se utiliza um dicionário que associa palavras a escores numéricos de polaridade ou intensidade emocional (*SentiLex*, português, *VADER*, inglês etc.).

- Exemplo: na frase “O atendimento **foi excelente**”, o sistema deve identificar um sentimento positivo; já em “O serviço **não foi dos melhores**”, ele deve identificar um sentimento negativo.

PRINCIPAIS TAREFAS EM PLN

Reconhecimento de Entidades Nomeadas (NER)

Reconhecimento de entidades nomeadas, ou **NER** (*Named Entity Recognition*), consiste em identificar, em uma sequência textual, trechos **que correspondem a entidades do mundo real** e classificá-los em categorias predefinidas. Trata-se de um problema de rotulação sequencial, no qual cada *token* recebe um rótulo segundo um esquema de anotação (como PER, para pessoa, ORG, para a organização, LOC, para local, entre outros).

- Exemplo: no texto “Maria trabalha na Petrobras em São Paulo”, o sistema identifica “Maria” como PER, “Petrobras” como ORG e “São Paulo” como LOC, estruturando essa informação em campos que podem alimentar uma base de conhecimento.

PRINCIPAIS TAREFAS EM PLN

Modelagem de Tópicos

A modelagem (ou identificação) de tópicos visa descobrir **temas recorrentes em uma coleção de documentos**, sem rótulos prévios. É, em geral, uma **abordagem tipicamente não supervisionada**, pois identifica padrões de coocorrência de termos e relações de significado ao longo dos textos analisados. O resultado são grupos temáticos caracterizados por conjuntos de palavras representativas e por documentos associados a esses temas.

- Exemplo: em milhares de comentários sobre eleições, o modelo pode identificar temas recorrentes como “corrupção”, “economia” e “segurança pública”, sem que essas categorias tenham sido previamente definidas. A partir disso, pode-se aplicar análise de sentimento (nesse caso, uma tarefa supervisionada) para avaliar a polaridade das opiniões em cada tema.

PRINCIPAIS TAREFAS EM PLN

Sumarização (saída = sequência textual)

Sumarização automática consiste em **gerar uma representação condensada de um texto** preservando seu conteúdo informacional central e sua coerência discursiva. Pode ser **extrativa**, quando seleciona e reorganiza trechos do próprio texto original com base em critérios de relevância, ou **abstrativa**, quando produz novas sentenças que sintetizam o conteúdo por meio de geração de linguagem. Em termos de formulação, pode ser supervisionada (treinada com pares texto–resumo) ou baseada em métodos não supervisionados de ranqueamento e relevância semântica.

- Exemplo: dado um artigo científico extenso, o sistema deve gerar um resumo que apresente problema, método e principais resultados, mantendo fidelidade factual e eliminando redundâncias.

PRINCIPAIS TAREFAS EM PLN

Geração de Texto (saída = sequência textual)

Consiste em **produzir novos conteúdos textuais a partir de uma instrução, pergunta ou contexto inicial**, modelando a geração como previsão sequencial de tokens condicionada ao histórico fornecido. O objetivo é gerar sequências coerentes, gramaticalmente corretas e semanticamente adequadas à solicitação. O avanço recente dessa área está associado aos **LLMs** (*Large Language Models*), baseados em arquiteturas **Transformer** e treinados em larga escala com aprendizado auto-supervisionado.

- Exemplo: ao receber a instrução “Explique, em dois parágrafos, o que é PLN”, o sistema deve gerar uma resposta clara e consistente, respeitando o formato pedido.

PRINCIPAIS TAREFAS EM PLN

Tradução Automática (saída = sequência textual)

A tradução automática consiste em **converter um texto de um idioma para outro**, preservando seu sentido e produzindo uma **saída natural na língua de destino**. É uma das tarefas mais abrangentes do PLN porque integra simultaneamente desambiguação lexical, dependência de contexto, modelagem sintática e adequação pragmática. O sistema precisa interpretar corretamente o significado na língua de origem e reconstruí-lo de forma equivalente na língua de chegada, evitando traduções literais inadequadas.

- Exemplo: traduzir “This exam was a piece of cake” exige que o sistema reconheça que a expressão é idiomática e significa “Essa prova foi muito fácil”, e não “Essa prova foi um pedaço de bolo”. Aqui o desafio envolve identificar linguagem figurada e produzir uma equivalência natural no idioma de destino.

PRINCIPAIS TAREFAS EM PLN

Geração de Legendas (saída = sequência textual)

Consiste em **produzir, a partir do áudio de um vídeo, uma transcrição sincronizada em texto, geralmente com marcação temporal**. Em sua forma mais comum, envolve reconhecimento automático de fala para converter áudio em texto. É tipicamente formulada como um problema supervisionado, treinado com grandes volumes de pares áudio–transcrição. A qualidade é frequentemente avaliada por métricas como taxa de erro de palavras, além de critérios de sincronização temporal.

- Exemplo: em um vídeo educacional, o sistema gera automaticamente legendas alinhadas ao tempo de fala do professor, permitindo acessibilidade para pessoas com deficiência auditiva ou para usuários que assistem sem áudio.

PRINCIPAIS TAREFAS EM PLN

Perguntas e Respostas (Q&A)

Sistemas de perguntas e respostas recebem uma **pergunta em linguagem natural e produzem uma resposta**, normalmente com base em um contexto (documentos, trechos ou base de conhecimento). Em geral é supervisionado, treinado com pares pergunta–resposta ou com dados anotados de evidência.

- Exemplo: dado um trecho que contém “Marie Curie ganhou dois prêmios Nobel, em Física e em Química”, a pergunta “Quantos prêmios Nobel Marie Curie ganhou?” deve ser respondida com “Dois”, idealmente indicando como evidência o trecho “ganhou dois prêmios Nobel”.

PRINCIPAIS TAREFAS EM PLN

Reconhecimento de Fala (multimodal)

Converte sinal de áudio da fala humana em texto, enfrentando variações como ruído, sotaque, ritmo, entonação e coarticulação. Em geral, é **treinado de forma supervisionada**, utilizando **grandes volumes de áudio alinhados a transcrições humanas**. A avaliação costuma empregar métricas como taxa de erro de palavras (*Word Error Rate - WER*).

- Exemplo: transcrever uma frase falada rapidamente, distinguindo “para” de “pra” e tratando hesitações, pausas e repetições de modo consistente com o objetivo do sistema.

PRINCIPAIS TAREFAS EM PLN

Descrição de Imagens (multimodal)

Consiste em **gerar uma descrição textual em linguagem natural a partir do conteúdo visual de uma imagem**. Trata-se de uma tarefa multimodal, pois combina visão computacional (extração de características visuais) com PLN. Em geral, é formulada como um problema supervisionado, utilizando pares imagem–descrição para treinar modelos capazes de mapear representações visuais para sequências textuais coerentes. Arquiteturas modernas frequentemente utilizam modelos baseados em **redes neurais profundas** com mecanismos de atenção ou **transformers** multimodais.

- Exemplo: dada uma imagem contendo “uma mulher atravessando a rua com um guarda-chuva em um dia chuvoso”, o sistema deve produzir uma descrição equivalente em texto. É especialmente relevante para tecnologias assistivas voltadas a pessoas com deficiência visual.

CONCEITOS

Em Processamento de Linguagem Natural, trabalha-se tipicamente com **conjuntos textuais definidos para análise**, denominados *corpus*. O termo é preferido por indicar um conjunto selecionado com critérios metodológicos, diferentemente da expressão genérica “dados”.

CORPUS

Conjunto de textos selecionado intencionalmente, com recorte analítico claro e finalidade definida.

CORPORA

Plural acadêmico de *corpus*, utilizado quando se comparam diferentes conjuntos textuais.



II – PROCESSO DE ANÁLISE DE DADOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)

PROCESSO DE ANÁLISE DE DADOS

Independentemente da técnica, projetos de análise de dados costumam seguir um conjunto recorrente de etapas que vão do entendimento do problema à geração de *insights* acionáveis.

Esse processo foi formalizado sob o termo **KDD** (*Knowledge Discovery in Databases*) em 1996 para descrever a **descoberta de conhecimento** por meio de **etapas** como seleção, pré-processamento, transformação, **mineração** e interpretação.

Essa mesma lógica foi posteriormente estruturada em metodologias operacionais como o **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), amplamente adotado para organizar e conduzir **projetos de mineração de dados** em ambientes organizacionais. As etapas eram *Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, Deployment*.

PROCESSO DE ANÁLISE DE DADOS

Vamos adotar um processo analítico genérico, organizado em **seis macroetapas** proposto em processos de treinamento de certificação oficial da Google*:

- *Entendimento*
- *Preparação*
- *Pré-processamento*
- *Modelagem*
- *Compartilhamento*
- *Ação*

ENTENDIMENTO

Nesta etapa definimos qual fenômeno linguístico queremos investigar e qual tarefa de PLN será formulada. Diferentemente de análises tabulares, aqui a pergunta precisa ser traduzida em termos de linguagem: que tipo de texto será analisado, qual comportamento textual se deseja identificar e qual saída o sistema deverá produzir.

Isso envolve explicitar se o objetivo é, por exemplo, **classificar documentos, medir sentimento, responder perguntas** etc.

A **pergunta analítica** passa a ser formulada como uma tarefa sobre a linguagem, com definição clara da unidade observacional, do tipo de inferência desejada e do critério de sucesso da análise.

PREPARAÇÃO

A etapa de preparação antecede qualquer tratamento computacional do texto. Seu foco é definir **qual material linguístico será utilizado e em que condições ele será analisado**.

Aqui decidimos quais **fontes textuais** fazem sentido para o problema proposto, qual recorte será considerado, qual unidade textual será adotada (por exemplo, documentos, sentenças ou interações) e se o conjunto disponível é adequado para responder à pergunta analítica.

Isso inclui atividades como identificar repositórios ou sistemas de origem, selecionar coleções documentais, verificar cobertura e representatividade do *corpus*, avaliar volume de dados, analisar condições de acesso e **critérios de inclusão ou exclusão de textos**.

PRÉ-PROCESSAMENTO

Pré-processar dados textuais significa preparar o corpus para análise linguística, assegurando que o conjunto de documentos seja representativo, coerente e tecnicamente utilizável. O objetivo não é apenas integridade formal, mas garantir que o material reflita adequadamente o fenômeno linguístico estudado, evitando distorções introduzidas por **ruídos de coleta ou problemas de origem**.

Incluem-se aqui atividades como remoção de documentos duplicados ou quase duplicados, eliminação de ruído estrutural típico de textos extraídos da web (**menus, anúncios, marcações HTML**), correção de problemas de codificação de caracteres, **filtragem de textos truncados** ou fora do escopo analítico, verificação e padronização de metadados e organização do *corpus* em formato adequado para processamento computacional.

MODELAGEM

Agora analisamos o *corpus* já processado para responder à pergunta analítica por meio de técnicas de PLN.

O foco deixa de ser a organização do material e passa a ser sua representação analítica, isto é, a **transformação da linguagem em estruturas** que permitam identificar padrões, relações e comportamentos linguísticos relevantes.

Entram nesta fase as operações próprias do *pipeline* de PLN, como segmentação textual, normalização linguística, construção de **representações vetoriais** e geração de atributos derivados do texto, além da aplicação de métodos estatísticos ou de aprendizado de máquina quando necessários.

COMPARTILHAMENTO

Compartilhar é comunicar os resultados da modelagem textual de forma clara e adequada ao público que utilizará o sistema ou interpretará seus achados.

Em projetos de PLN, isso pode envolver explicar como o texto foi representado, quais tarefas foram executadas (classificação, extração, sumarização, geração etc.), quais métricas foram utilizadas para avaliação e quais limitações o sistema apresenta.

A forma de apresentação é decisiva. É necessário contextualizar o *corpus* utilizado, explicitar possíveis vieses linguísticos, delimitar escopo de aplicação e esclarecer limites do modelo diante de ambiguidade, variações de contexto e mudanças no uso da linguagem.

AÇÃO

Agir, em projetos de PLN, consiste em **aplicar o sistema desenvolvido em um contexto real de uso**. Isso pode significar integrar um classificador a um fluxo de atendimento, incorporar um mecanismo de extração de entidades a um sistema jurídico, utilizar um modelo de sentimento para monitoramento de opinião ou implantar um sistema de geração ou resposta automática em ambiente produtivo.

Assim como em outros projetos analíticos, a etapa de ação inclui acompanhamento contínuo. Sistemas de PLN são sensíveis a mudanças linguísticas, novos termos, gírias, variações culturais e alterações no domínio textual. Portanto, esta etapa envolve monitorar desempenho, revisar *corpus* quando necessário, atualizar modelos e avaliar impacto prático das decisões automatizadas. O ciclo permanece iterativo: novas falhas observadas ou novos usos da linguagem podem demandar ajustes no modelo e reinício do processo analítico.



III - PIPELINE PLN ENGENHARIA DE ATRIBUTOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)

PIPELINE EM PLN

Chamamos de *pipeline* o encadeamento de etapas técnicas executadas dentro da macroetapa **Analisar**, nas quais os textos são transformados em representações computacionais e utilizados para especificar, **treinar, validar e avaliar modelos de PLN**.

É esse fluxo que detalharemos a seguir, a saber:

- * Engenharia de atributos
- Especificação do modelo
- Treinamento
- Validação
- Teste

* Utilizamos a expressão “Engenharia de atributos” como um **conceito guarda-chuva** para designar qualquer transformação orientada ao modelo, isto é, decisões que moldam os dados para torná-los adequados a alguma tarefa específica de PLN.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

Engenharia de atributos (*feature engineering*) é o processo de criar, transformar, selecionar ou agregar variáveis a partir dos dados brutos com o **objetivo de melhorar a capacidade do modelo**.

As etapas específicas variam conforme o tipo de dado, mas o princípio é o mesmo: transformar informação bruta em representação útil ao modelo. Em **PLN**, isso envolve:

- **Segmentação** (definir unidades linguísticas de análise)
 - **Normalização textual** (reduzir variações formais não informativas)
 - **Extração de informação** (estruturar elementos linguísticos relevantes)
 - **Representação vetorial do texto** (converter texto em espaço numérico modelável)
- Em **PLN**, o dado bruto é **texto**, que não é diretamente modelável. O texto não nasce estruturado como variáveis prontas para modelagem. Portanto, a *feature engineering* envolve estruturar a própria representação do dado antes da geração de atributos derivados.
 - Em PLN **feature construction** e **feature selection** são frequentemente efeitos das etapas anteriores, enquanto em dados tabulares elas são etapas explícitas e separadas

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Tokenização

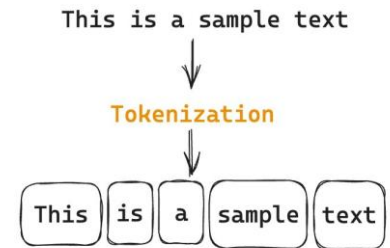
Tokenização é o processo de “**segmentar o texto em unidades básicas** chamadas ***tokens***”, que servirão como entrada para o modelo. Ela define qual será a **unidade de análise linguística** e, portanto, influencia o tipo de padrão que o modelo poderá aprender.

- Por exemplo, na **análise de sentimento**, costuma-se utilizar tokenização por **palavras**, pois o objetivo é capturar termos como “excelente”, “péssimo” ou “horrível”. Em tarefas como descrição de imagens e geração de legendas, modelos modernos baseados em ***Transformers*** utilizam tokenização por **subpalavras**.

Obs.: não há uma regra fixa associada à tarefa. A decisão técnica depende do modelo adotado e da estratégia analítica.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Tokenização por Palavras

Consiste em segmentar o texto em unidades correspondentes a **palavras**, delimitadas por espaços ou sinais de pontuação segundo regras linguísticas ou heurísticas.

- Exemplo: “O sistema falhou novamente.”
Tokens: [“O”, “sistema”, “falhou”, “novamente”]

É a abordagem típica do PLN clássico, sendo amplamente utilizada em representações como: Bag-of-Words, TF-IDF.

Seu principal benefício é a **interpretabilidade**, pois o modelo opera diretamente sobre **palavras explícitas do vocabulário**.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Tokenização por Subpalavras

Consiste em segmentar palavras em **unidades menores que uma palavra** com base na sua **frequência estatística** dentro no **corpus**.

- Exemplo: “infelizmente”
Tokens possíveis: [“in”, “feliz”, “mente”]

Essa abordagem é dominante em modelos modernos baseados em ***Transformers***, como: BERT, RoBERTa e GPT.

Seu principal benefício é reduzir o problema de palavras fora do vocabulário, controlar o tamanho do vocabulário e permitir melhor generalização para palavras raras ou compostas. É adequada quando se trabalha com grandes modelos neurais e vocabulário aberto (caso moderno dos LLMs.).

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Tokenização por palavra (BoW, TF-IDF)

Corpus grande → facilmente 500 mil, 1 milhão ou mais palavras distintas. Cada uma vira um token diferente.

- Resultado:
Vocabulário enorme.
Muitas palavras raras.
Muitas “palavras nunca vistas”.

```
feliz  
infeliz  
infelizmente  
felicidade  
felicíssimo
```

Tokenização por subpalavra (BERT, RoBERTa e GPT)

O vocabulário é fixado previamente (BERT, 30k tokens, GPT 50K tokens) e aprendido estatisticamente a partir do corpus.

```
feliz  
##mente  
in
```

```
in + feliz + ##mente
```

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Tokenização por Caracteres

Consiste em segmentar o texto em **caracteres individuais**.

- Exemplo: “falha”
Tokens: [“f”, “a”, “l”, “h”, “a”]

É utilizada em abordagens específicas que buscam robustez a erros ortográficos, variações morfológicas ou quando se deseja modelar padrões muito finos da linguagem. É menos frequente como estratégia principal em arquiteturas modernas de grande escala.

As sequências tornam-se maiores e o custo computacional aumenta.

PIPELINE EM PLN

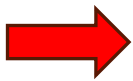
ENGENHARIA DE ATRIBUTOS



Normalização

Consiste em aplicar “transformações ao texto com o objetivo de **reduzir variações que não agregam distinção relevante**” para a tarefa de modelagem. O objetivo não é alterar o conteúdo semântico, mas tornar a representação textual mais consistente, reduzindo redundâncias e variações morfológicas que poderiam aumentar artificialmente a dimensionalidade do espaço de atributos.

- Em termos práticos, a normalização pode envolver **remoção de palavras** pouco informativas, **redução de palavras** às suas formas básicas e **padronização de formas** linguísticas.
- Em PLN, algumas transformações associadas à *Preparação* também podem ser tratadas como orientadas ao modelo, tais como: *lowercasing*, remoção de **acentos**, tratamento de **pontuação**, correção **ortográfica** e normalização de **números** (datas, valores etc.). Isso porque, em PLN, a aplicação depende da tarefa e da **sensibilidade do modelo a essas variações** (“Varejo”, “varejo” ou “VAREJO”, “Premium”, “Premiun” ou “prêmio”).



PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Normalização – Remoção de Stopwords

Consiste em eliminar **palavras muito frequentes** que, em determinadas tarefas, contribuem pouco para diferenciar documentos. Geralmente incluem artigos, preposições e conjunções.

- Exemplo : “O sistema não está funcionando corretamente.”
Após remoção de stopwords: [“sistema”, “funcionando”, “corretamente”]

É comum em representações clássicas como *Bag-of-Words* e TF-IDF, especialmente em classificação de textos e identificação de tópicos.

Seu objetivo é reduzir **ruído estatístico e dimensionalidade**, mantendo termos com maior poder informativo.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Normalização – Stemming

Stemming consiste em **reduzir palavras ao seu radical** por meio de regras heurísticas que removem sufixos e terminações (“truncam”).

- Exemplo: “caminhando”, “caminhou”, “caminhada”
Após stemming: “**caminh**”

O radical obtido pode não corresponder a uma palavra válida no idioma, mas permite agrupar variações morfológicas sob uma mesma base. É utilizado principalmente em abordagens clássicas para reduzir **variação lexical e dimensionalidade**.

Seu objetivo é tratar flexões como **manifestações de um mesmo núcleo semântico**.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Normalização – Lematização

Lematização consiste em reduzir palavras à sua forma canônica (**lema**), considerando a **morfologia flexional**.

- Exemplo: “falando”, “falou”, “falei”, “falaram”
Após lematização: “**falar**”

Diferentemente do *stemming*, a lematização **preserva validade linguística e realiza análise morfológica**.

gênero (bonito / bonita)

número (carro / carros)

tempo verbal (amou / amando / amarão)

Atenção: a lematização **não** trata **derivação, prefixação** ou **sufixação**.



derivação (feliz ≠ felicidade ≠ infelizmente)
prefixação (feliz ≠ infeliz)
sufixação produtiva

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Extração de Informação

Consiste em **identificar e estruturar elementos linguísticos relevantes** presentes no texto, transformando conteúdo não estruturado em representações organizadas e interpretáveis.

O objetivo é explicitar padrões que não estão formalmente delimitados no texto bruto, como **categorias gramaticais, entidades, relações ou estruturas sintáticas**.

Diferentemente da normalização, que **reduz** variações formais, a extração de informação **adiciona estrutura analítica ao texto**, tornando explícitos componentes que podem ser utilizados como **atributos para modelos mais complexos**.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

Extração de Informação – POS Tagging

POS (*Part-of-Speech*) consiste em atribuir a cada palavra **sua classe gramatical**, como substantivo, verbo, adjetivo ou advérbio.

- Exemplo: “O sistema falhou novamente.”
Anotação possível: O (DET), sistema (NOUN), falhou (VERB), novamente (ADV)

Essa técnica adiciona estrutura gramatical explícita ao texto, permitindo analisar padrões sintáticos e gerar atributos derivados, como proporção de substantivos ou frequência de verbos.

Seu objetivo é **estruturar a dimensão morfossintática** do texto.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS



Extração de Informação – Named Entity Recognition (NER)

Consiste em **identificar e classificar entidades nomeadas no texto**, como pessoas, organizações, locais, datas ou valores monetários.

- Exemplo: “Maria trabalha na Petrobras desde 2020.”

NER: Maria → Pessoa, Petrobras → Organização, 2020 → Data

NER pode ser o objetivo final da tarefa (extração automática de entidades) ou uma etapa intermediária para geração de atributos, como contagem de entidades ou identificação de atores relevantes.

Seu objetivo é **estruturar elementos explícitos** presentes no discurso.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

Extração de Informação – Parsing Sintático

Parsing sintático consiste em **identificar** as **relações gramaticais** entre as palavras de uma **sentença**, construindo uma estrutura que representa **dependências sintáticas**.

- Exemplo: “O sistema apresentou falhas.”
apresentou → verbo principal,
sistema → sujeito de apresentou,
falhas → objeto de apresentou

Tecnicamente, o **parsing sintático** é a implementação computacional da **análise sintática** tradicional da linguística. Ela explicita como os termos se relacionam na estrutura da frase, permitindo **análises mais profundas de significado e relações**.

Seu objetivo é **estruturar a organização sintática da sentença**.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

1 2
3 4

Representação Vetorial

Converter texto em vetores ou matrizes para uso por modelos. Como os modelos operam sobre **vetores e matrizes**, o texto precisa ser transformado **preservando padrões linguísticos relevantes**.

Em síntese, representar vetorialmente é transformar linguagem em **geometria computacional**.

A representação define **o que será preservado** no texto:

- **Frequência lexical (contagem de termos)**
- **Coocorrência estatística (padrões de contexto)**
- **Similaridade semântica (relações semânticas)**

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

Representação Vetorial
Por Frequência Lexical
(**BoW** e **TF-IDF**)

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

Representação Vetorial – Frequência Lexical

Esparsas (clássicas): vetores de **alta dimensionalidade** baseados em contagem/peso de termos (BoW, TF-IDF). O significado é representado pela presença e frequência explícita de termos no documento. Existe interpretabilidade lexical.

Documentos são caracterizados por distribuição lexical explícita, não por estrutura semântica latente. Similaridade é baseada em sobreposição de vocabulário, não em proximidade semântica (carro *versus* automóvel). Dois documentos são considerados próximos se **compartilham as mesmas palavras**.

Palavras são tratadas como unidades independentes cuja importância decorre de sua **frequência ou raridade no corpus**.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

Representação Vetorial – One-Hot Encoding (básico)

Cada **token** é representado por um vetor binário do tamanho do vocabulário. A representação é **esparsa** (quase todas as posições são 0, com um único 1 por *token*) e de alta dimensionalidade (cada vetor tem dimensão $|V|$).

- Se o vocabulário é composto por: **modelo, análise, dados, sistema, algoritmo**, temos a seguinte representação **one-hot**:

modelo $\rightarrow [1, 0, 0, 0, 0]$

análise $\rightarrow [0, 1, 0, 0, 0]$

dados $\rightarrow [0, 0, 1, 0, 0]$

sistema $\rightarrow [0, 0, 0, 1, 0]$

algoritmo $\rightarrow [0, 0, 0, 0, 1]$

Imagine o tamanho dos vetores para um $|V| = 10000$ tokens?

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

Representação Vetorial – BoW (clássico)

O **Bag-of-Words** representa o **documento inteiro** como um único vetor de dimensão $|V|$ (tamanho do vocabulário). Cada posição indica a **frequência** de um **token** no documento (dessa forma ignorando a ordem das palavras). A representação é esparsa e de alta dimensionalidade.

- Vocabulário: **modelo, análise, dados, sistema, algoritmo**
Texto: “um modelo de análise de dados é um modelo inteligente”
Representação no **BoW**: [2, 1, 1, 0, 0]
- Ou seja, 2 ocorrências de “modelo”, 1 de “análise”, 1 de “dados”, nenhuma de “sistema” ou “algoritmo”. A posição do elemento no vetor indica o *token* no vocabulário e a quantidade de vezes em que ele aparece no documento.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

O que vira um vetor no BoW?

No BoW, cada unidade de análise vira um vetor.

Essa unidade pode ser:

- um documento inteiro (mais comum)
- um parágrafo
- uma frase
- um título
- um comentário
- um e-mail
- uma notícia

Depende da tarefa.

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

1 2
3 4

Representação Vetorial – TF-IDF (clássico)

O *Term Frequency – Inverse Document Frequency* também representa cada documento como um vetor de dimensão $|V|$, ponderando a frequência de cada token pela sua **raridade no corpus**. Enquanto o BoW considera só contagem bruta, o TF-IDF reduz o peso de termos muito frequentes no conjunto de documentos e aumenta o peso de termos mais informativos. A representação continua sendo esparsa e de alta dimensionalidade.

- Formalmente: $\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$

- onde:

$\text{TF}(t, d)$ = frequência do token t no documento d

$\text{IDF}(t) = \log (N / \text{df}(t))$

N = # total de documentos

$\text{df}(t)$ = # de documentos que contêm o token t

- Se um termo aparece em quase todos os documentos, seu peso diminui.
- Se aparece em poucos documentos, seu peso aumenta.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

TF-IDF(t, d) = TF(t, d) × IDF(t), ou

$$w_{i,j} = t f_{i,j} \times \log \left(\frac{N}{df_i} \right)$$

Onde:

- $tf_{i,j}$ = número de ocorrências de i em j
- df_i = número de documentos contendo i
- N = número total de ocorrências

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	2/7	2/7	$\log(2/2) = 0$	0	0
Car	1/7	0	$\log(2/1) = 0.3$	0.043	0
Truck	0	1/7	$\log(2/1) = 0.3$	0	0.043
Is	1/7	1/7	$\log(2/2) = 0$	0	0
Driven	1/7	1/7	$\log(2/2) = 0$	0	0
On	1/7	1/7	$\log(2/2) = 0$	0	0
The	1/7	1/7	$\log(2/2) = 0$	0	0
Road	1/7	0	$\log(2/1) = 0.3$	0.043	0
Highway	0	1/7	$\log(2/1) = 0.3$	0	0.043

Condições:

A: *The car is driven on the road.* (O carro é conduzido na estrada)

B: *The truck is driven on the highway.* (O caminhão é conduzido na rodovia)

O valor calculado para TF-IDF das palavras comuns aos documentos foi igual a zero (**menos significativas**). Por outro lado, o valor para as palavras “car”, “truck”, “road” e “highway” é maior (**mais significativas**).

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

TF-IDF(t, d) = TF(t, d) × IDF(t), ou

$$w_{i,j} = tf_{i,j} \times \log \left(\frac{N}{df_i} \right)$$

onde:

$tf_{i,j}$ = frequência do termo i no documento j

N = número total de documentos no corpus

df_i = número de documentos que contêm o termo i

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	2/7	2/7	$\log(2/2) = 0$	0	0
Car	1/7	0	$\log(2/1) = 0.3$	0.043	0
Truck	0	1/7	$\log(2/1) = 0.3$	0	0.043
Is	1/7	1/7	$\log(2/2) = 0$	0	0
Driven	1/7	1/7	$\log(2/2) = 0$	0	0
On	1/7	1/7	$\log(2/2) = 0$	0	0
Road	1/7	0	$\log(2/1) = 0.3$	0.043	0
Highway	0	1/7	$\log(2/1) = 0.3$	0	0.043

A: *The car is driven on the road.* (O carro é conduzido na estrada)

B: *The truck is driven on the highway.* (O caminhão é conduzido na rodovia)

O valor calculado para TF-IDF das palavras comuns aos documentos foi igual a zero (**menos significativas**). Por outro lado, o valor para as palavras “car”, “truck”, “road” e “highway” é maior (**mais significativas**).

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

A: the car is driven on the road

B: the truck is driven on the highway

C: the man is walking on the floor

$$w_{i,j} = tf_{i,j} \times \log \left(\frac{N}{df_i} \right)$$

onde:

$tf_{i,j}$ = frequência do termo i no documento j

N = número total de documentos no corpus

df_i = número de documentos que contêm o termo i

O vetor TF-IDF destaca **as palavras que diferenciam os documentos**, ignorando os termos muito comuns.

Term	TF(A)	TF(B)	TF(C)	df	IDF	TF-IDF(A)	TF-IDF(B)	TF-IDF(C)
the	2/7	2/7	2/7	3	0	0	0	0
is	1/7	1/7	1/7	3	0	0	0	0
on	1/7	1/7	1/7	3	0	0	0	0
driven	1/7	1/7	0	2	0.176	0.025	0.025	0
car	1/7	0	0	1	0.477	0.068	0	0
road	1/7	0	0	1	0.477	0.068	0	0
truck	0	1/7	0	1	0.477	0	0.068	0
highway	0	1/7	0	1	0.477	0	0.068	0
man	0	0	1/7	1	0.477	0	0	0.068
walking	0	0	1/7	1	0.477	0	0	0.068
floor	0	0	1/7	1	0.477	0	0	0.068

(0.176 = $\log_{10}(3/2)$; 0.477 = $\log_{10}(3/1)$; 0.025 = $(1/7)*0.176$; 0.068 = $(1/7)*0.477$)

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

N-gramas (unigrama, bigrama e trigrama)

N-gramas são sequências de **n tokens** consecutivos extraídas de um texto. O objetivo é **capturar padrões locais de linguagem** que envolvem uma ou mais palavras. Dependendo do valor de n , a sequência representa diferentes níveis de contexto:

Unigram (1-gram) - Considera cada token individualmente.

Bigram (2-gram) - Considera pares de tokens consecutivos.

Trigram (3-gram) - Considera trios de tokens consecutivos.

▪ Exemplo: *eu gosto de ciência de dados*

Unigrama: *eu | gosto | de | ciência | de | dados*

Bigrama: *eu gosto | gosto de | de ciência | ciência de | de dados*

Trigrama: *eu gosto de | gosto de ciência | de ciência de | ciência de dados*

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

N-gramas em Bag-of-Words (BoW)

Cada palavra ou sequência de palavras se torna um atributo do vetor. Os **n-grams** definem quais sequências serão contabilizadas.

Texto:

eu gosto de dados

BoW com unigram

termo	frequência
eu	1
gosto	1
de	1
dados	1

BoW com bigram

termo	frequência
eu gosto	1
gosto de	1
de dados	1

BoW com Trigram

termo	frequência
eu gosto de	1
gosto de dados	1

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

N-gramas em TF-IDF

No TF-IDF, os **n-grams também definem atributos**, mas em vez de simples contagem o método aplica um **peso estatístico** baseado na **frequência no documento** e na **raridade no corpus**.

- **Exercício - Considere o seguinte corpus com três documentos:**

D1: eu gosto de dados

D2: eu gosto de ciência

D3: dados e ciência

- Utilizando **bigramas ($n = 2$)**:
 - Liste os **bigramas gerados em cada documento**.
 - Calcule o **DF** de cada bigrama.
 - Calcule o **IDF** de cada bigrama.
 - Calcule o **TF-IDF para os bigramas do documento D1**.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

D1: eu gosto de dados
D2: eu gosto de ciência
D3: dados e ciência

1. Bigrams

D1 → eu gosto, gosto de, de dados
D2 → eu gosto, gosto de, de ciência
D3 → dados e, e ciência

3. DF de cada bigrama

bigrama	DF
eu gosto	2
gosto de	2
de dados	1
de ciência	1
dados e	1
e ciência	1

4. IDF de cada bigrama

Usando $IDF = \log(3/DF)$

bigrama	IDF
eu gosto	$\log(3/2)$
gosto de	$\log(3/2)$
de dados	$\log(3)$
de ciência	$\log(3)$
dados e	$\log(3)$
e ciência	$\log(3)$

2. TF por documento

bigrama	TF em D1	TF em D2	TF em D3
eu gosto	1	1	0
gosto de	1	1	0
de dados	1	0	0
de ciência	0	1	0
dados e	0	0	1
e ciência	0	0	1

5. TF-IDF para D1

bigrama	TF em D1	IDF	TF-IDF
eu gosto	1	$\log(3/2) = 0,405$	0,405
gosto de	1	$\log(3/2) = 0,405$	0,405
de dados	1	$\log(3) = 1,099$	1,099

PIPELINE EM PLN ENGENHARIA DE ATRIBUTOS

Representação Vetorial Por Coocorrência Estatística (**PMI**)

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

Representação Vetorial – Coocorrência Estatística

Baseia-se em **quantas vezes palavras aparecem próximas umas das outras no corpus**, utilizando uma **janela de contexto**. Essas relações são organizadas em **matrizes**, nas quais cada palavra é representada pelos termos que costumam aparecer ao seu redor.

Técnicas como **PMI** exploram essas matrizes para identificar padrões estatísticos no uso das palavras.

A similaridade passa a depender **dos contextos compartilhados**, e não apenas da presença das mesmas palavras. Assim, palavras que aparecem em **contextos semelhantes** tendem a ser representadas de forma parecida, revelando **relações semânticas aproximadas derivadas da estrutura estatística da linguagem**.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

PMI – Pointwise Mutual Information

PMI é uma medida usada em modelos de representação vetorial baseados em **coocorrência de palavras**. A técnica parte de uma **matriz palavra-contexto**, que registra quantas vezes uma palavra aparece **próxima de outra** dentro de um *corpus*.

A medida central é o **PMI (*Pointwise Mutual Information*)**, que quantifica o quanto a coocorrência entre uma palavra e um contexto é **maior ou menor do que seria esperado pelo acaso**. Sua função é medir **quão fortemente uma palavra está associada a um contexto**, comparando a coocorrência observada com a coocorrência esperada ao acaso.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

PMI – Pointwise Mutual Information

Se duas palavras aparecem juntas com frequência maior que a esperada pela **probabilidade independente** de cada uma, o valor de PMI será positivo.

PMI (Pointwise Mutual Information)

Mede quanto duas palavras aparecem juntas mais (ou menos) do que seria esperado ao acaso.

$$PMI(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$

Interpretação:

- $PMI > 0$ → associação positiva (aparecem juntas mais do que o esperado)
- $PMI = 0$ → independência
- $PMI < 0$ → aparecem juntas menos do que o esperado

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

PPMI – Positive Pointwise Mutual Information

O problema do PMI é que ele também pode assumir **valores negativos**, indicando que duas palavras aparecem juntas menos do que o esperado. Em representações vetoriais de linguagem, esses **valores negativos não são úteis**.

Por isso utiliza-se o **PPMI (Positive PMI)**, que simplesmente substitui valores negativos por zero:

$$PPMI(w, c) = \max(PMI(w, c), 0)$$

Apenas **associações positivas entre palavras e contextos** são preservadas. O resultado é uma matriz de representação vetorial mais adequada para capturar **relações semânticas** entre palavras em modelos baseados em coocorrência.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

Probabilidade Conjunta e Independência de Eventos

Considere o experimento de lançar **duas moedas consecutivamente**.
Calcule:

- (a) a **probabilidade** de a 1ª moeda dar cara;
- (b) a **probabilidade** de a 1ª moeda dar cara e a 2ª também dar cara.

Eventos independentes

$$P(A, B) = P(A) \times P(B)$$

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

12
34

Probabilidade Conjunta e Dependência de Eventos

Considere o experimento de **retirar duas cartas consecutivamente de um baralho, sem reposição**. Calcule:

- (a) a probabilidade de a 1ª carta ser um Ás;
- (b) a probabilidade de a 1ª carta ser um Ás e a 2ª também ser um Ás.

Eventos dependentes

$$P(A, B) = P(A) \times P(B|A)$$

Se A influencia B, então:

$$P(B|A) \neq P(B)$$

Ou seja, a ocorrência de A altera a probabilidade de B.

PIPELINE EM PLN

ENGENHARIA DE ATRIBUTOS

1 2
3 4

Probabilidade em um corpus

Em PLN, consideramos eventos como:

- (a) A: aparecer a **palavra w**;
- (b) A: aparecer a **palavra c**;

Probabilidade marginal

$$P(w) = \frac{\text{freq}(w)}{N}$$

$$P(c) = \frac{\text{freq}(c)}{N}$$

Probabilidade conjunta

$$P(w, c) = \frac{\text{cooc}(w, c)}{N}$$

onde:

- $\text{cooc}(w, c)$ = número de vezes que as palavras aparecem juntas
- N = total de ocorrências consideradas

Ideia central: duas palavras são independentes quando a probabilidade de ocorrerem juntas é igual ao produto de suas probabilidades marginais, isto é, $P(w, c) = P(w) \times P(c)$

BOW

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]  
  
[('bebe', 5),  
 ('come', 5),  
 ('gato', 4),  
 ('cachorro', 4),  
 ('agua', 3),  
 ('carne', 3),  
 ('leite', 2),  
 ('peixe', 2),  
 ('homem', 2)]
```

bow

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
D1	1	1	0	0	0	1	0	0	0
D2	1	1	1	0	0	0	0	0	0
D3	0	1	0	0	0	1	0	1	0
D4	0	1	1	0	0	0	0	1	0
D5	0	0	0	0	1	1	0	0	1
D6	0	0	1	0	1	0	0	0	1
D7	0	0	0	1	1	1	0	0	0
D8	0	0	1	1	1	0	0	0	0
D9	1	1	0	0	0	0	1	0	0
D10	0	0	0	1	1	0	1	0	0

TF-IDF

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]
```

```
[('bebe', 5),  
 ('come', 5),  
 ('gato', 4),  
 ('cachorro', 4),  
 ('agua', 3),  
 ('carne', 3),  
 ('leite', 2),  
 ('peixe', 2),  
 ('homem', 2)]
```

tfidf

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
D1	0.641771	0.512414	0.000000	0.000000	0.000000	0.570581	0.000000	0.000000	0.000000
D2	0.641771	0.512414	0.570581	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
D3	0.000000	0.482845	0.000000	0.000000	0.000000	0.537655	0.000000	0.691222	0.000000
D4	0.000000	0.482845	0.537655	0.000000	0.000000	0.000000	0.000000	0.691222	0.000000
D5	0.000000	0.000000	0.000000	0.000000	0.482845	0.537655	0.000000	0.000000	0.691222
D6	0.000000	0.000000	0.537655	0.000000	0.482845	0.000000	0.000000	0.000000	0.691222
D7	0.000000	0.000000	0.000000	0.641771	0.512414	0.570581	0.000000	0.000000	0.000000
D8	0.000000	0.000000	0.570581	0.641771	0.512414	0.000000	0.000000	0.000000	0.000000
D9	0.582818	0.465343	0.000000	0.000000	0.000000	0.000000	0.666168	0.000000	0.000000
D10	0.000000	0.000000	0.000000	0.582818	0.465343	0.000000	0.666168	0.000000	0.000000

PMI

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]
```

bigrams_list

```
[('gato', 'bebe'),  
 ('bebe', 'agua'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'agua'),  
 ('gato', 'bebe'),  
 ('bebe', 'leite'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'leite'),  
 ('gato', 'come'),  
 ('come', 'peixe'),  
 ('cachorro', 'come'),  
 ('come', 'peixe'),  
 ('gato', 'come'),  
 ('come', 'carne'),  
 ('cachorro', 'come'),  
 ('come', 'carne'),  
 ('homem', 'bebe'),  
 ('bebe', 'agua'),  
 ('homem', 'come'),  
 ('come', 'carne')]
```

bigram_freq

```
Counter({('gato', 'bebe'): 2,  
         ('bebe', 'agua'): 3,  
         ('cachorro', 'bebe'): 2,  
         ('bebe', 'leite'): 2,  
         ('gato', 'come'): 2,  
         ('come', 'peixe'): 2,  
         ('cachorro', 'come'): 2,  
         ('come', 'carne'): 3,  
         ('homem', 'bebe'): 1,  
         ('homem', 'come'): 1})
```

Calcular total de coocorrências.

N = matrix.values.sum()

N

np.int64(20)

p_wc = matrix / N

p_wc

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
bebe	0.15	0.00	0.0	0.00	0.00	0.0	0.0	0.1	0.0
cachorro	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
carne	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
come	0.00	0.00	0.0	0.15	0.00	0.0	0.0	0.0	0.1
gato	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
homem	0.00	0.05	0.0	0.00	0.05	0.0	0.0	0.0	0.0
leite	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
peixe	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0

PMI

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]
```

bigrams_list

```
[('gato', 'bebe'),  
 ('bebe', 'agua'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'agua'),  
 ('gato', 'bebe'),  
 ('bebe', 'leite'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'leite'),  
 ('gato', 'come'),  
 ('come', 'peixe'),  
 ('cachorro', 'come'),  
 ('come', 'peixe'),  
 ('gato', 'come'),  
 ('come', 'carne'),  
 ('cachorro', 'come'),  
 ('come', 'carne'),  
 ('homem', 'bebe'),  
 ('bebe', 'agua'),  
 ('homem', 'come'),  
 ('come', 'carne')]
```

bigram_freq

```
Counter({'gato', 'bebe'): 2,  
        ('bebe', 'agua'): 3,  
        ('cachorro', 'bebe'): 2,  
        ('bebe', 'leite'): 2,  
        ('gato', 'come'): 2,  
        ('come', 'peixe'): 2,  
        ('cachorro', 'come'): 2,  
        ('come', 'carne'): 3,  
        ('homem', 'bebe'): 1,  
        ('homem', 'come'): 1})
```

Calcular total de coocorrências.

N = matrix.values.sum()

N

np.int64(20)

p_w

	$\emptyset$
agua	0.00
bebe	0.25
cachorro	0.20
carne	0.00
come	0.25
gato	0.20
homem	0.10
leite	0.00
peixe	0.00

p_c

	$\emptyset$
agua	0.15
bebe	0.25
cachorro	0.00
carne	0.15
come	0.25
gato	0.00
homem	0.00
leite	0.10
peixe	0.10

p_wc = matrix / N

p_wc

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
bebe	0.15	0.00	0.0	0.00	0.00	0.0	0.0	0.1	0.0
cachorro	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
carne	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
come	0.00	0.00	0.0	0.15	0.00	0.0	0.0	0.0	0.1
gato	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
homem	0.00	0.05	0.0	0.00	0.05	0.0	0.0	0.0	0.0
leite	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
peixe	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0

100%

1

```
[('gato', 'bebe'),
 ('bebe', 'agua'),
 ('cachorro', 'bebe'),
 ('bebe', 'agua'),
 ('gato', 'bebe'),
 ('bebe', 'leite'),
 ('cachorro', 'bebe'),
 ('bebe', 'leite'),
 ('gato', 'come'),
 ('come', 'peixe'),
 ('cachorro', 'come'),
 ('come', 'peixe'),
 ('gato', 'come'),
 ('come', 'carne'),
 ('cachorro', 'come'),
 ('come', 'carne'),
 ('homem', 'bebe'),
 ('bebe', 'agua'),
 ('homem', 'come'),
 ('come', 'carne')]
```

$$PMI(w,c) = \log(P(w,c) / (P(w) \times P(c)))$$

$P(w,c)$ = probabilidade conjunta do par palavra-contexto

$P(c)$ = probabilidade marginal do contexto

```
Counter({'gato', 'bebe': 2,
        ('bebe', 'agua'): 3,
        ('cachorro', 'bebe'): 2,
        ('bebe', 'leite'): 2,
        ('gato', 'come'): 2,
        ('come', 'peixe'): 2,
        ('cachorro', 'come'): 2,
        ('come', 'carne'): 3,
        ('homem', 'bebe'): 1,
        ('homem', 'come'): 1})
```

[illegible]

PROBLEMAS DAS REPRESENTAÇÕES BASEADAS EM FREQUÊNCIA/COOCORRÊNCIA

1) Ambiguidade (Polissemia)

Problema: uma mesma palavra pode ter significados diferentes.

- Exemplo:
- “Ele sentou no banco.”
- “Ele foi ao banco.”

Em **BoW** e **TF-IDF**, “banco” é apenas um termo do vocabulário.

O modelo não entende sentidos (só conta **quantas vezes** cada palavra aparece).

PROBLEMAS DAS REPRESENTAÇÕES BASEADAS EM FREQUÊNCIA/COOCORRÊNCIA

2) Variabilidade (Sinonímia)

Problema: palavras diferentes com mesmo significado.

- Exemplo:

Doc A: “O carro é rápido.”

Doc B: “O automóvel é rápido.”

Doc C: “ O veículo é veloz.”

Tanto para **BoW** quanto para o **TF-IDF**: “carro” $\neq$ “automóvel” $\neq$ “veículo”.

O modelo não entende sinônimos (só conta **quantas vezes** cada palavra aparece).

PROBLEMAS DAS REPRESENTAÇÕES BASEADAS EM FREQUÊNCIA/COOCORRÊNCIA

3) Dependência de Contexto (Relações Semânticas Latentes)

Problema: textos podem falar do mesmo tema com outras palavras.

- Exemplo:

Doc A: “Ele comprou ações na bolsa de valores.”

Doc B: “O mercado financeiro caiu devido à crise.”

Doc C: “O urso tomou conta do mercado.”

Os textos falam sobre finanças. Mas não compartilham palavras iguais.

Em **BoW** e **TF-IDF**, a similaridade será baixa, pois o modelo só compara palavras idênticas. Ele não entende que “ações”, “bolsa” e “mercado financeiro” pertencem ao mesmo campo de significado.



IV - PIPELINE PLN ESPECIFICAÇÃO DE MODELOS

Prof. Luis Claudio

M.Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)

ESPECIFICAÇÃO DE MODELOS CLÁSSICOS MAIS USADOS EM PLN

Supervisionados Clássicos

- Regressão Logística
- Naive Bayes
- SVM (*Support Vector Machine*)

Não Supervisionado Clássicos

- LDA (*Latent Dirichlet Allocation*)
- K-Means

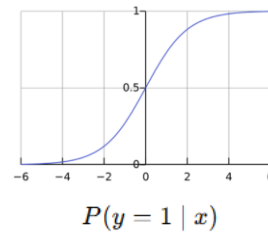
ESPECIFICAÇÃO DE MODELOS CLÁSSICOS POUCO USADOS EM PLN

Outros Modelos menos Usados em PLN

Regressão Linear - O problema central é a incompatibilidade de saída. A regressão linear prevê valores numéricos contínuos e infinitos, enquanto o PLN lida quase exclusivamente com categorias discretas (classificação). Tentar aplicar uma reta em dados textuais resulta em previsões sem sentido matemático (como valores negativos para uma categoria), além de o modelo ser extremamente sensível a outliers causados pela frequência disparatada de certas palavras em um corpus.

KNN (*K-Nearest Neighbors*) - Sofre severamente com a "maldição da dimensionalidade". Como o texto vetorizado (TF-IDF) gera matrizes com milhares de colunas, o cálculo de distância euclidiana torna-se computacionalmente caríssimo e perde a precisão, pois em espaços de alta dimensão, todos os pontos parecem estar à mesma distância uns dos outros. Para ser minimamente útil, exigiria funções de similaridade de cosseno, mas ainda assim seria inviável para produção devido à lentidão na busca por vizinhos em grandes bases.

Árvores de Decisão e Random Forest - Embora potentes para dados tabulares, esses modelos não lidam bem com a esparsidade extrema característica do texto. Em uma matriz onde 99% das entradas são zero, as árvores tendem a criar ramificações excessivamente profundas e complexas para tentar isolar variáveis, o que gera modelos pesados (em memória e disco) e com alta tendência ao overfitting, performando geralmente pior que modelos lineares simples como a Regressão Logística.

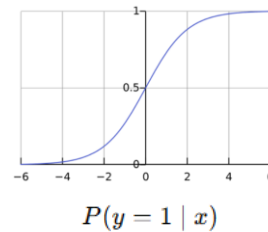


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

No aprendizado supervisionado, modelos são treinados a partir de dados rotulados ($X \rightarrow y$), com o objetivo de **aprender a relação entre variáveis explicativas e a variável alvo** para realizar previsões.

Essa abordagem se divide em dois tipos de problema: **regressão**, quando o objetivo é prever **valores contínuos** (y contínuo), e **classificação**, quando o objetivo é prever **classes/categorias** (y discreto).

No contexto de classificação em PLN, a regressão logística é um dos modelos mais utilizados, devido à sua simplicidade (:o), eficiência e boa **interpretabilidade**.

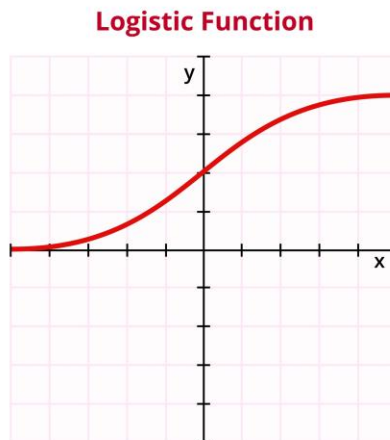


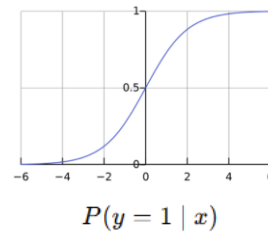
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Em **PLN**, a classificação de texto é uma das tarefas mais comuns em PLN (sentimento, autoria, spam etc.). Nestes casos, a **regressão logística** é utilizada para estimar a **probabilidade de uma observação pertencer à classe 1**, dado um conjunto de atributos.

Para isso, o modelo transforma uma saída linear em uma **probabilidade entre 0 e 1** por meio de uma **função específica** (*sigmoide*, para binária, *softmax* para classes múltiplas).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

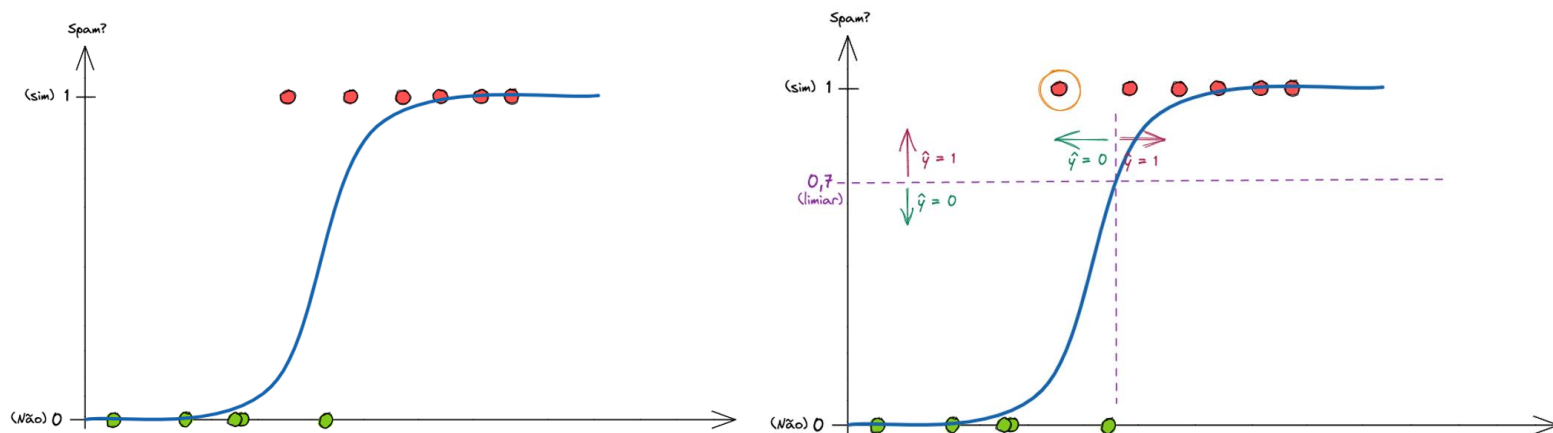




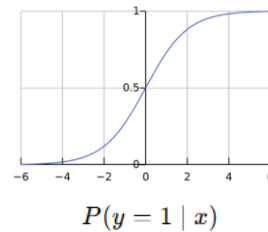
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Regra de decisão

O modelo retorna uma probabilidade $p = \sigma(z)$ para a classe positiva. Para obter a classificação, define-se um limiar, geralmente 0.5.



Esse limiar não é obrigatório (pode ser ajustado conforme o custo do erro). Deixar passar fraude ou bloquear compra legítima? Posso definir limiar em 0.3, e ser conservador; ou 0.7, e ser agressivo.



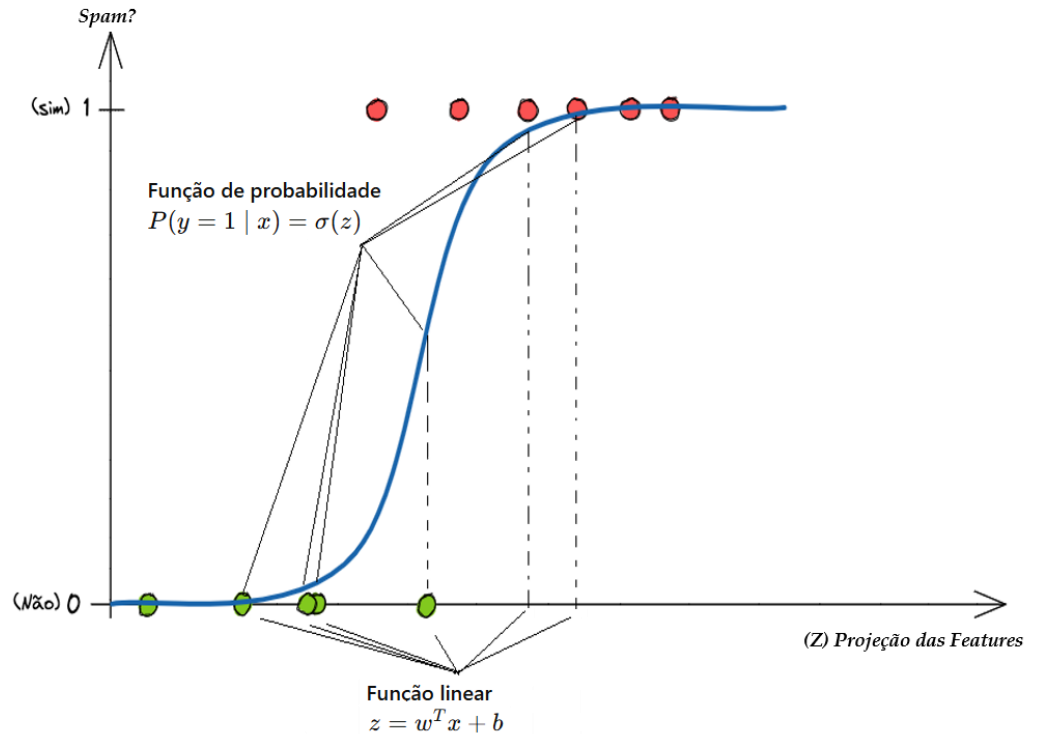
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Regra de decisão

Os pontos no gráfico representam os **dados reais** do conjunto de treino, onde cada observação está **rotulada como 0 ou 1** (por exemplo, não spam ou spam). Ou seja, representam os **rótulos**.

Durante o treinamento, o modelo ajusta seus parâmetros (w , b) para gerar a “melhor” curva sigmoide representa a probabilidade de um registro **pertencer à classe 1**.

A curva **não separa** **perfeitamente os pontos**.





ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Quando usar?

Classificação de texto

Representação BoW / TF-IDF (alta dimensionalidade esparsa)

Quando precisa de **baseline forte e interpretável**

Cenários com *features* **altamente esparsas**

Vantagens

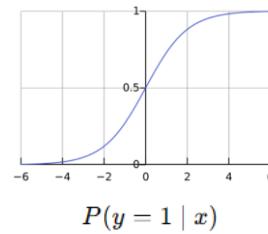
Simple, rápido e estável

Coefficientes interpretáveis (peso das palavras)

Observação:

É possível utilizar regressão logística com representações baseadas em *embeddings*?

Sim, porém, por ser um **modelo linear**, ela não explora totalmente relações não lineares presentes em *embeddings* mais ricos (especialmente contextuais).

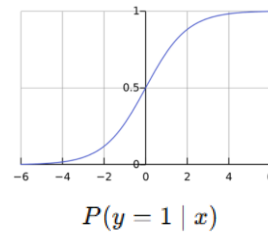


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

A regressão logística clássica utiliza a função **sigmoide** (função logística) para problemas binários. Para problemas multiclasse, utiliza-se uma generalização desse modelo baseada na função **softmax**

O modelo calcula inicialmente uma **combinação linear** das variáveis de entrada, cujo resultado pode variar de $-\infty$ a $+\infty$, e em seguida aplica uma função de transformação para convertê-lo em probabilidades. No caso binário, utiliza-se a função sigmoide; no caso multiclasse, a função softmax.

$$z = w \cdot X + b \quad \longrightarrow \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

X representa o vetor de entrada (*features*) do modelo.

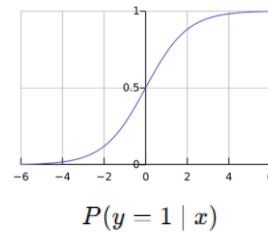
Passo (1) - Durante o treinamento, o modelo utiliza dados rotulados (X, y) para ajustar os parâmetros w (pesos) e b (viés). Com esses parâmetros, o modelo calcula a combinação linear:

$$z = w^T x + b$$

É esse valor z que pode variar de $-\infty$ a $+\infty$.

Passo (2) - Calculado o valor de z , o próximo passo é calcular, via função sigmoide $\sigma(z)$, a probabilidade entre 0 e 1 utilizada na decisão de classe.

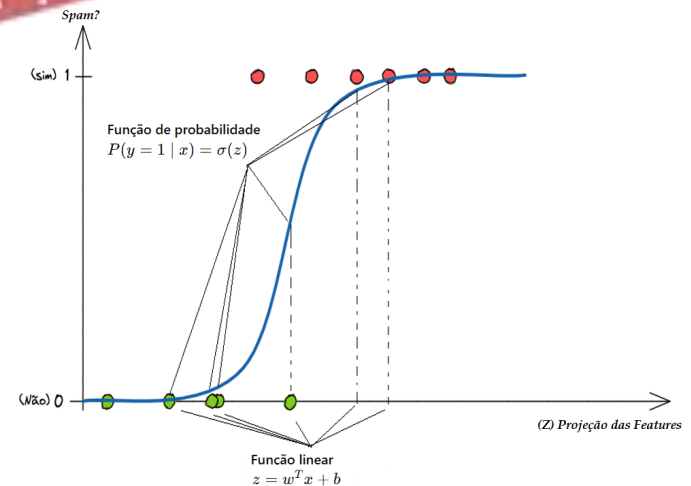
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

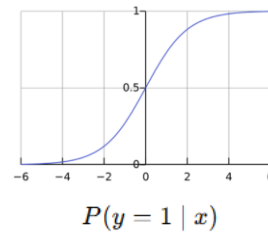


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Os parâmetros w (pesos) e b (viés) são **ajustados durante o treinamento** por meio de um processo de **otimização** que **minimiza** o erro entre as previsões do **modelo** e os **valores reais**. O detalhamento desse **processo** não será **abordado neste momento**.

$$(w, b) = \arg \min \sum_i [-y_i \log(\sigma(w^T x_i + b)) - (1 - y_i) \log(1 - \sigma(w^T x_i + b))]$$





ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

$|V|$: [excelente, ruim]

Suponha que o modelo já aprendeu:

- Peso (w) da palavra “**excelente**” = **+2**
- Peso (w) da palavra “**ruim**” = **-2**
- Viés (b) = **0**

Pesos aprendidos e viés:

$$w = \begin{bmatrix} 2 \\ -2 \end{bmatrix} \quad b = 0$$

Para o texto “o produto é excelente”:

$$X = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

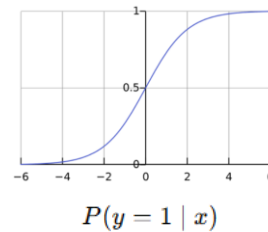
Cálculo para “o produto é excelente”:

$$z = w^T X + b = \begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0 \quad z = w^T x + b$$

$$z = (2 \cdot 1) + (-2 \cdot 0) = 2$$

$$\sigma(2) = \frac{1}{1 + e^{-2}} \approx 0.88$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

$|V|$: [excelente, ruim]

Suponha que o modelo já aprendeu:

- Peso (w) da palavra “**excelente**” = +2
- Peso (w) da palavra “**ruim**” = -2
- Viés (b) = 0

Pesos aprendidos e viés:

$$w = \begin{bmatrix} 2 \\ -2 \end{bmatrix} \quad b = 0$$

Para o texto “o produto é ruim”:

$$X = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Cálculo para “o produto é ruim”:

$$z = w^T X + b = \begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0$$

$$z = w \cdot X + b$$

$$z = (2 \cdot 0) + (-2 \cdot 1) = -2$$

$$\sigma(-2) = \frac{1}{1 + e^2} \approx 0.12$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Teorema de Bayes

Thomas Bayes foi um matemático e estatístico britânico, conhecido pelo teorema que leva seu nome. O Teorema de Bayes é um princípio fundamental da probabilidade, utilizado para atualizar **probabilidades à medida que novas evidências são observadas**.

A ideia central é permitir calcular a **probabilidade de um evento dado uma evidência** por meio de uma forma alternativa. Quando essa probabilidade é difícil de obter diretamente, o teorema permite reescrevê-la a partir de probabilidades mais acessíveis, como a **probabilidade da evidência dado o evento**.

$$P(y | x) = \frac{(P(x | y) \cdot P(y))}{P(x)} \quad \text{(Teorema de Bayes)}$$

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Teorema de Bayes

Ou seja, às vezes temos $P(x | y)$, mas queremos $P(y | x)$.

Nesses casos, usa Bayes.

Por exemplo:

Medicina: pode ser difícil estimar a probabilidade de uma pessoa ter a doença dado que o teste foi positivo, enquanto é mais fácil estimar a probabilidade de o teste dar positivo dado que a pessoa tem a doença.

Segurança (alarme): pode ser difícil estimar a probabilidade de haver invasão dado que o alarme tocou, enquanto é mais fácil estimar a probabilidade de o alarme tocar dado que houve invasão.

PLN: é difícil estimar a probabilidade de um texto ser spam dado o conjunto de palavras, enquanto é mais fácil estimar a probabilidade do conjunto de palavras aparecer dado que o texto é spam.

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Teorema de Bayes (exemplo 1 – teste doença)

Sabe-se que uma doença atinge **20%** das pessoas. Existe um teste para essa doença. Sabe-se que, entre as pessoas que **têm a doença**, **80%** recebem resultado positivo. Foram testadas todas as **1000 pessoas** de um vilarejo e, ao todo, **320 pessoas** tiveram resultado positivo. Pergunta: se uma pessoa deste vilarejo fez o teste e deu positivo, qual a probabilidade de ela realmente ter a doença?

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Teorema de Bayes (exemplo 2 – spam)

Sabe-se que **20%** dos e-mails são spam. Existe um critério baseado na presença de determinadas palavras no texto. Sabe-se que, entre os e-mails que são spam, **80% contêm essas palavras**. Foram analisados todos os **1000 e-mails** de uma base e, ao todo, **320 e-mails** contêm essas palavras. Pergunta: se um e-mail dessa base contém essas palavras, qual a probabilidade de ele realmente ser spam?

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Teorema de Bayes (exemplo 3 – spam)

Suponha que queremos classificar um e-mail como spam ou não spam. Sabemos que **20% dos e-mails são spam**. Além disso, a palavra “free” aparece em **70%** dos e-mails classificados como **spam** e em **10%** dos e-mails classificados como **não spam**.

Dado um novo e-mail com a palavra “free”, qual é a probabilidade de ele ser spam?

Passo 1 — Aplicar a fórmula

$$P(\text{spam} \mid \text{free}) = \frac{P(\text{free} \mid \text{spam}) \cdot P(\text{spam})}{P(\text{free})}$$

Passo 2 — Substituir valores

$$P(\text{spam} \mid \text{free}) = \frac{0,7 \cdot 0,2}{P(\text{free})}$$

Passo 3 — Calcular $P(\text{free})$

$$P(\text{free}) = (0,7 \cdot 0,2) + (0,1 \cdot 0,8)$$

$$P(\text{free}) = 0,14 + 0,08 = 0,22$$

Passo 4 — Resultado final

$$P(\text{spam} \mid \text{free}) = \frac{0,14}{0,22} \approx 0,64$$

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Naive Bayes

O Naive Bayes é um modelo de classificação supervisionada que utiliza o Teorema de Bayes assumindo que as variáveis são **independentes entre si**.

A principal simplificação do modelo é justamente essa **suposição de independência**, o que permite calcular as probabilidades de forma simples e eficiente. Na prática, o modelo “age como se” as palavras não dependessem umas das outras.

Essa suposição não é realista, pois **o significado de um texto depende do contexto**, porém, tal suposição simplifica muito os cálculos. O termo “**naive**” (**ingênuo**) refere-se exatamente a essa simplificação, que torna o modelo viável e eficiente na prática.

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Naive Bayes (forma geral)

No problema de classificação de textos, queremos estimar a **probabilidade de uma classe dado um texto**. Ou seja, queremos calcular $P(c | x)$, onde c representa a classe (por exemplo, spam ou não spam) e x representa o conjunto de palavras do texto.

Como essa probabilidade é difícil de estimar diretamente, utilizamos o Teorema de Bayes para reescrevê-la a partir de probabilidades mais acessíveis. Assim, obtemos:

$$P(c | x) \propto P(c) \cdot \prod P(w | c)$$

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Corpus

- Spam
"free win money"
"win free offer"
- Não spam
"meeting schedule"
"project meeting"

Contagens

- Total spam = 6 palavras
free=2, win=2, money=1, offer=1
- Total não spam = 4 palavras
meeting=2, schedule=1, project=1
- Vocabulário total = 7 palavras

$$P(w | c) = \frac{\text{count}(w, c) + 1}{\text{total de palavras em } c + |V|}$$

Probabilidades

- $P(\text{spam}) = 2/4 = 0,5$
- $P(\text{não spam}) = 2/4 = 0,5$
- $P(\text{free} | \text{spam}) = (2 + 1)/(6 + 7) = 3/13$
- $P(\text{win} | \text{spam}) = (2 + 1)/(6 + 7) = 3/13$
- $P(\text{free} | \text{não spam}) = (0 + 1)/(4 + 7) = 1/11$
- $P(\text{win} | \text{não spam}) = (0 + 1)/(4 + 7) = 1/11$

$$P(c | x) \propto P(c) \cdot \prod P(w | c)$$

Novo texto

"free win"

Cálculo

- Spam:

$$P(\text{spam}) \times P(\text{free} | \text{spam}) \times P(\text{win} | \text{spam})$$

$$= 0,5 \times \frac{3}{13} \times \frac{3}{13} \approx 0,0266$$

- Não spam:

$$P(\text{não spam}) \times P(\text{free} | \text{não spam}) \times P(\text{win} | \text{não spam})$$

$$= 0,5 \times \frac{1}{11} \times \frac{1}{11} \approx 0,0041$$

Resultado

- Spam: $\approx 0,0266$
- Não spam: $\approx 0,0041$

Como:

$$0,0266 > 0,0041$$

→ classificação: **spam**

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Naive Bayes

Em resumo, para cada novo documento, o modelo calcula a probabilidade de cada classe multiplicando a probabilidade da classe pelas probabilidades das palavras do texto, assumindo independência entre elas.

$$P(y) \times P(x_1|y) \times P(x_2|y) \times \cdots \times P(x_{50}|y)$$

Se, após o pré-processamento, o documento tiver 50 palavras, o cálculo do produto será feito considerando essas 50 palavras. O vocabulário total (ex: 10.000 palavras) é usado no treinamento e na suavização das probabilidades.



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - NAIVE BAYES

Quando usar?

Classificação de texto

Representação BoW / TF-IDF (alta dimensionalidade esparsa)

Quando precisa de **baseline muito rápido**

Cenários com *features* **altamente esparsas**

Problemas com **independência aproximada entre palavras**

Vantagens

Extremamente rápido (treino e inferência)

Robusto com poucos dados

Bom desempenho em tarefas simples (ex: spam)

ESPECIFICAÇÃO DE MODELOS

NAIVE BAYES VS REGRESSÃO LOGÍSTICA

Aspecto	Naive Bayes	Regressão Logística
Representação ideal	BoW / contagens	BoW / TF-IDF
Poucos dados	Muito bom	Razoável
Capacidade de modelar não linearidade	Muito limitada	Limitada (linear)
Desempenho geral	Bom (simples)	Melhor (em geral)
Suposição	Independência entre features	Sem suposição de independência
Alta dimensionalidade (esparsa)	Muito bom	Muito bom
Features esparsas	Muito bom	Muito bom
Uso com embeddings	Possível, mas conceitualmente fraco	Possível, mas não explora totalmente
Interpretabilidade	Média	Alta
Velocidade (treino/inferência)	Muito alta	Alta
Simplicidade	Muito alta	Alta
Exemplo de aplicação	Classificação de spam	Análise de sentimento

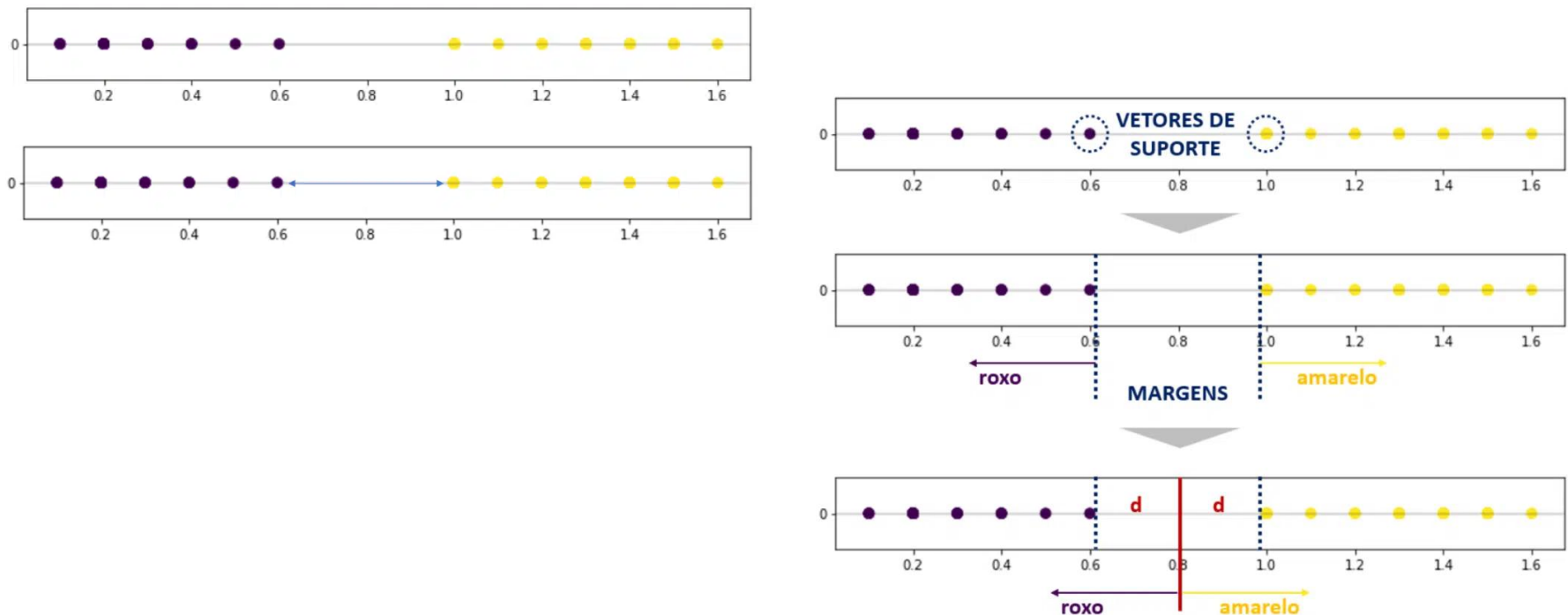
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

O **Support Vector Machine (SVM)** é um modelo de aprendizado supervisionado que busca encontrar o melhor **limite de separação** entre dados. Esse limite de separação (reta para duas dimensões, plano para três dimensões ou hiperplanos para mais de três dimensões), é definido de forma a **maximizar a margem entre as classes**, tornando o modelo mais robusto a variações nos dados

Diferentemente da regressão logística, que otimiza probabilidades associadas às classes e permite **interpretar o grau de confiança das previsões**, o SVM foca exclusivamente na **separação geométrica** dos dados, maximizando a margem entre as classes. Na prática, ambos podem produzir fronteiras lineares semelhantes em problemas de classificação de texto, mas a regressão logística oferece interpretação probabilística, enquanto o SVM prioriza robustez na separação.

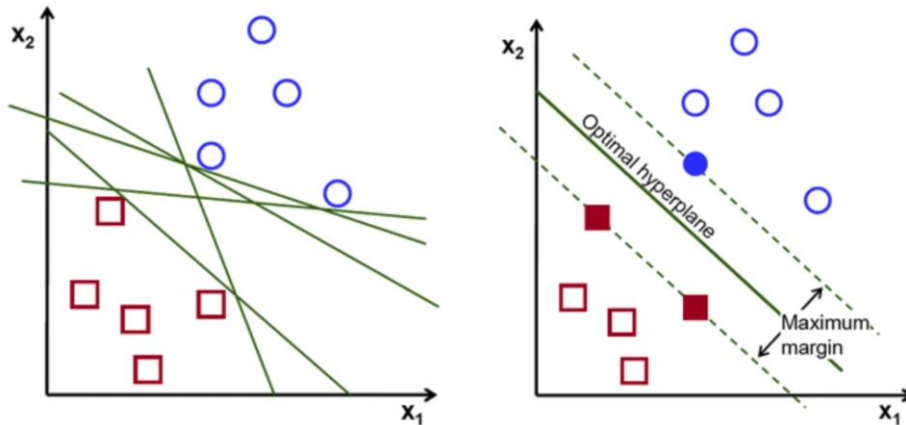
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

Os **vetores de suporte** são os pontos do conjunto de dados mais próximos da fronteira de decisão, sendo os principais responsáveis por defini-la.



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

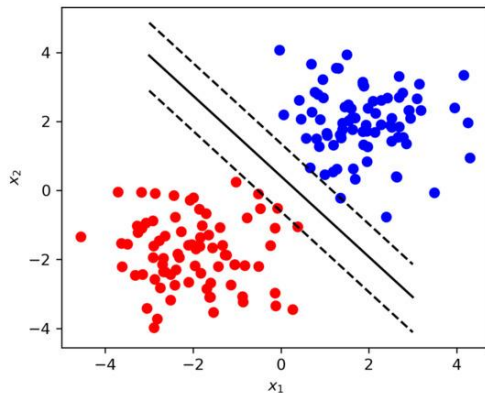
A partir desses pontos, o modelo define o **limite de separação** que maximiza a margem entre as classes, garantindo maior robustez e capacidade de generalização. Os demais pontos, mais distantes da fronteira, têm pouca influência no modelo.



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

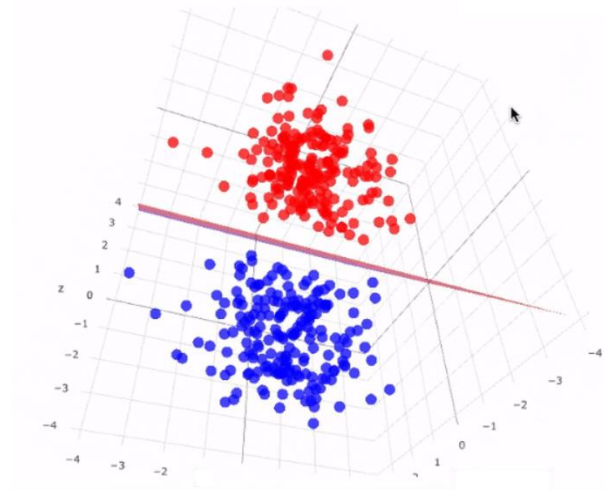
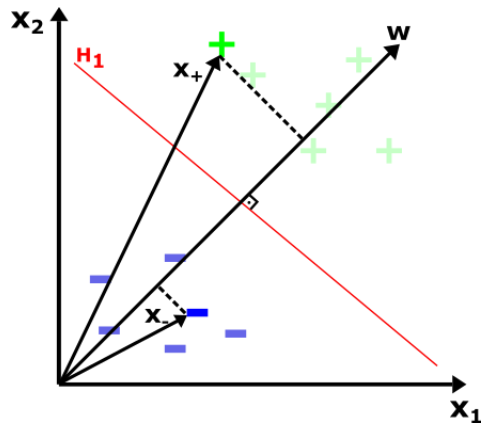
Exemplos 2D são apenas didáticos. Em PLN, trabalhamos com espaços de alta dimensionalidade (milhares de variáveis), especialmente com representações como **Bag of Words** e **TF-IDF**, onde cada termo do vocabulário (ou n-grama) se torna uma dimensão.

Em PLN, **raramente os dados são “perfeitamente” linearmente separáveis** (embora frequentemente sejam “aproximadamente” separáveis em espaços de alta dimensionalidade”).



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

Assim, o SVM é baseado na identificação dos **pontos mais críticos do conjunto de dados**, chamados **vetores de suporte**, que são aqueles mais próximos da fronteira de decisão.

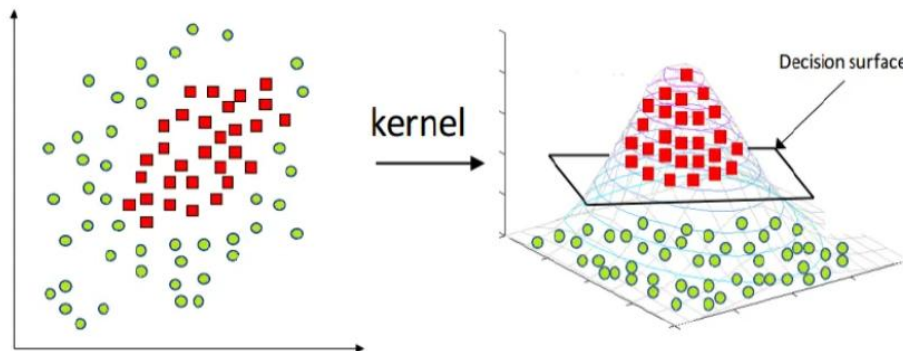


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

Kernel Trick

O SVM também pode ser aplicado tanto em dados não linearmente separáveis. Para isso, o modelo utiliza funções de kernel, que permitem **operar como se os dados estivessem em um espaço de maior dimensão**, onde a separação “linear” pode se tornar possível.

O chamado Kernel Trick é justamente esse mecanismo: em vez de transformar explicitamente os dados, o algoritmo calcula uma função de kernel que equivale a um **produto interno no espaço transformado**.

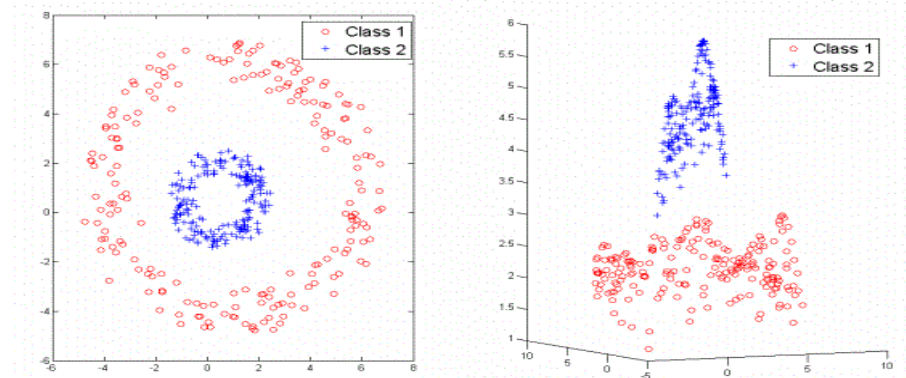


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

Kernel Trick

O SVM também pode ser aplicado tanto em dados não linearmente separáveis. Para isso, o modelo utiliza funções de kernel, que permitem **operar como se os dados estivessem em um espaço de maior dimensão**, onde a separação “linear” pode se tornar possível.

O chamado Kernel Trick é justamente esse mecanismo: em vez de transformar explicitamente os dados, o algoritmo calcula uma função de kernel que equivale a um **produto interno no espaço transformado**.



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

Funções de Kernel

Linear

$$K(x_i, x_j) = x_i \cdot x_j$$

- Mais usado em PLN
- Rápido e suficiente na maioria dos casos

RBF (Gaussian)

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

- Mais flexível
- Cria fronteiras complexas
- Muito usado fora de PLN

Polinomial

$$K(x_i, x_j) = (x_i \cdot x_j + c)^d$$

- Captura interações entre features
- Pode overfitar fácil

Sigmoid

$$K(x_i, x_j) = \tanh(\alpha x_i \cdot x_j + c)$$

- Similar a rede neural
- Menos comum na prática

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

Kernel Trick

Em PLN, **no entanto**, é comum utilizar o **kernel linear**, pois representações como Bag of Words e TF-IDF já projetam os dados em espaços de alta dimensionalidade (a separação linear costuma ser suficiente na prática).

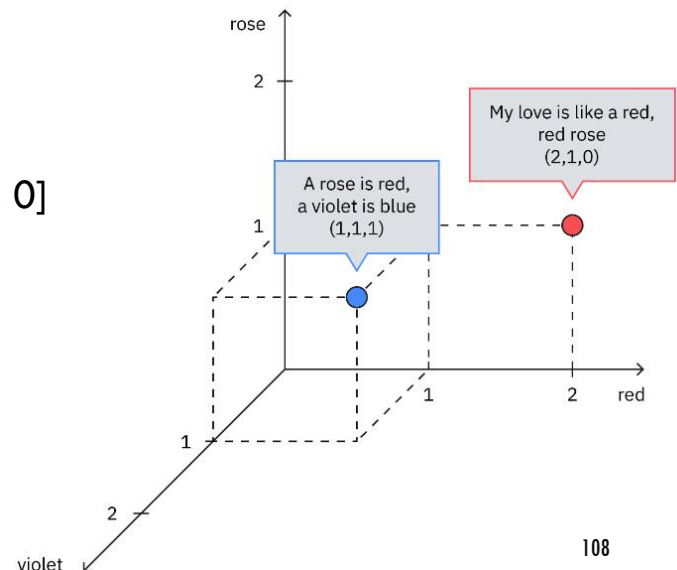
Exemplo simples (BoW com 3 palavras)

Vocabulário: [red, rose, violet]

Interpretação:

Documento 1: "A rose is red, a violet is blue" $\rightarrow (1, 1, 1)$

Documento 2: "My love is like a red, red rose" $\rightarrow [2, 1, 0]$





ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - SVM

Quando usar?

Classificação de texto (PLN)

Representação TF-IDF / BoW (alta dimensionalidade)

Dados aproximadamente separáveis

Cenários com dados de média escala

Quando busca maior margem de separação (robustez)

Vantagens

Bom desempenho em alta dimensionalidade

Eficaz mesmo com muitas features irrelevantes

Boa generalização (maximização da margem)

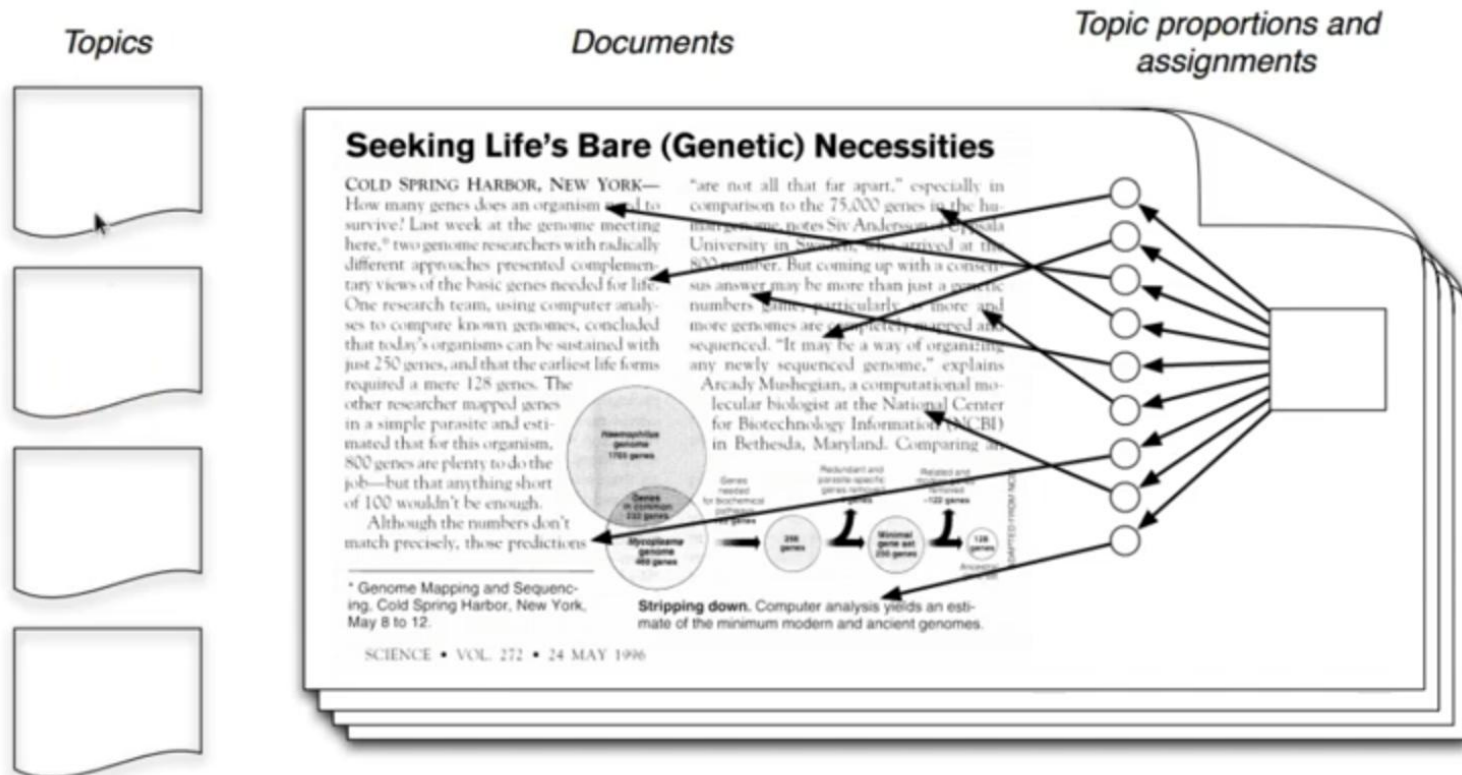
ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA

Latent Dirichlet Allocation (LDA) é um modelo probabilístico não supervisionado que identifica **tópicos latentes** em um conjunto de documentos. Cada documento é representado como uma **mistura de tópicos**, e cada tópico como uma **distribuição de palavras**.

Diferentemente de modelos de classificação, o **LDA não utiliza rótulos**, buscando **descobrir automaticamente padrões de coocorrência** de palavras que indicam a presença de diferentes temas nos textos.

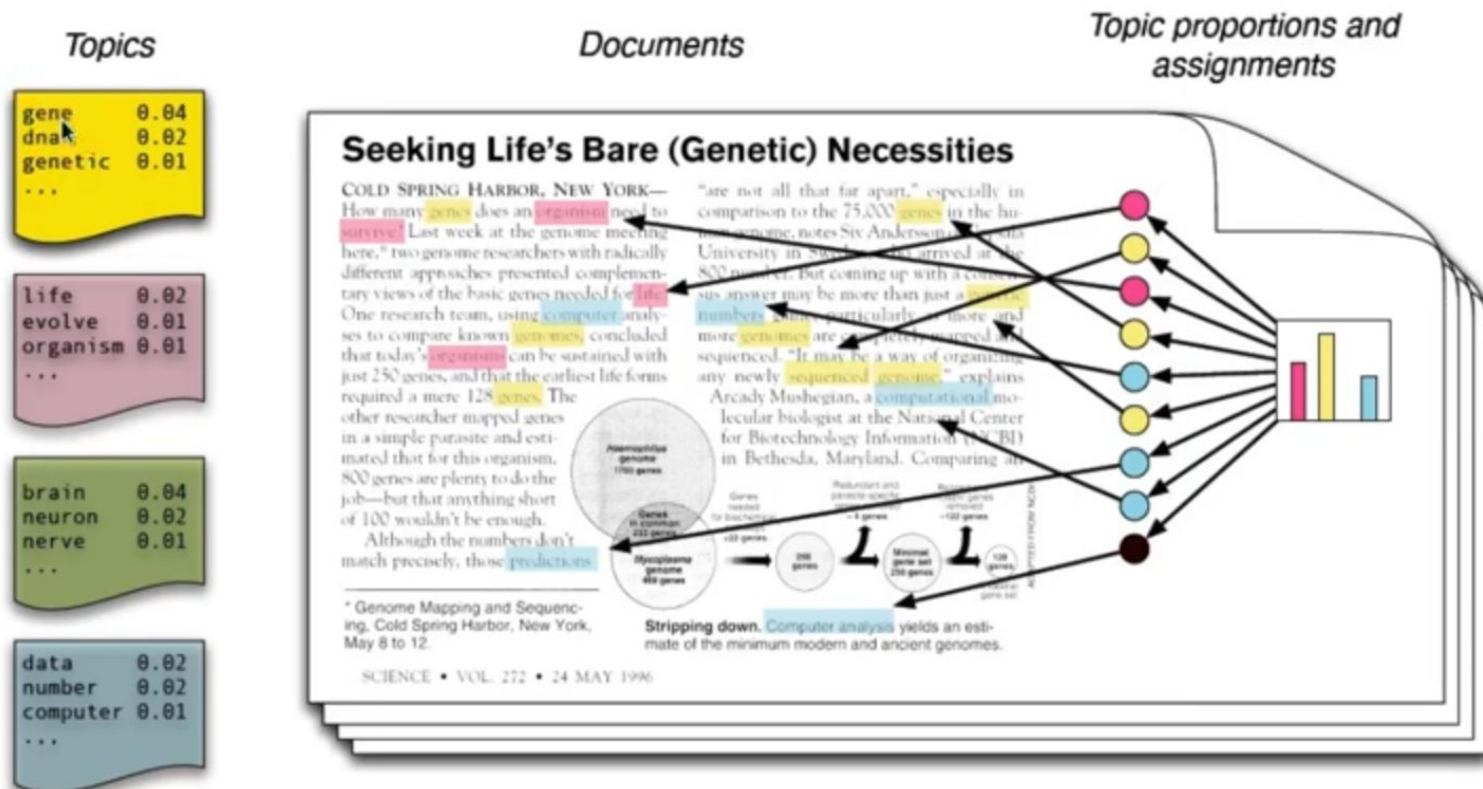
O LDA parte da ideia de que um documento pode abordar múltiplos assuntos simultaneamente. O objetivo do modelo é inferir essas estruturas ocultas a partir dos dados.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA



Cada **documento** é uma mistura de **tópicos**.
Cada **tópico** é uma distribuição de **palavras**.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA



Cada **documento** é uma mistura de **tópicos**.
Cada **tópico** é uma distribuição de **palavras**.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA

Tópico 1: depression (0.045), yong (0.017), treatments (0.017), thanks (0.014), best (0.014), overcome (0.013), hey (0.012), follow (0.012), video (0.009), good (0.009)
Tópico 2: like (0.022), love (0.015), people (0.013), just (0.010), depression (0.009), make (0.009), help (0.008), mental (0.008), look (0.007), need (0.007)
Tópico 3: im (0.050), dont (0.023), like (0.019), just (0.014), people (0.012), fuck (0.010), ive (0.009), know (0.008), love (0.008), trump (0.008)
Tópico 4: just (0.021), know (0.013), day (0.012), oh (0.011), want (0.010), amp (0.009), got (0.009), happy (0.008), dont (0.008), youre (0.008)
Tópico 5: thank (0.026), say (0.023), twitter (0.021), following (0.015), hello (0.014), good (0.014), way (0.011), girl (0.009), morning (0.009), anytime (0.009)

O modelo LDA acima identificou **5 tópicos globais** a partir do corpus. Cada tópico é representado por um conjunto de palavras com suas respectivas probabilidades $P(\text{palavra} | \text{tópico})$. Essas probabilidades indicam o quanto cada palavra é característica daquele tópico.

Note que alguns tópicos são **semanticamente coerentes**, como o Tópico 1 (saúde mental: *depression, treatments, overcome*); outros apresentam **palavras genéricas ou pouco informativas**, como *like, just, know*, indicando ruído nos dados.

É função do analista decidir **como ajustar o pipeline**, por exemplo: remover ou ampliar stopwords, aplicar filtros mais agressivos de frequência, revisar lematização/tokenização, reduzir ruído do corpus ou até alterar o número de tópicos e reexecutar o modelo.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA

Tópico 1: depression (0.045), yong (0.017), treatments (0.017), thanks (0.014), best (0.014), overcome (0.013), hey (0.012), follow (0.012), video (0.009), good (0.009)
Tópico 2: like (0.022), love (0.015), people (0.013), just (0.010), depression (0.009), make (0.009), help (0.008), mental (0.008), look (0.007), need (0.007)
Tópico 3: im (0.050), dont (0.023), like (0.019), just (0.014), people (0.012), fuck (0.010), ive (0.009), know (0.008), love (0.008), trump (0.008)
Tópico 4: just (0.021), know (0.013), day (0.012), oh (0.011), want (0.010), amp (0.009), got (0.009), happy (0.008), dont (0.008), youre (0.008)
Tópico 5: thank (0.026), say (0.023), twitter (0.021), following (0.015), hello (0.014), good (0.014), way (0.011), girl (0.009), morning (0.009), anytime (0.009)

Doc 1: It's just over 2 years since I was diagnosed with #anxiety and #depression. Today I'm taking a momen...
[0.01581274 0.57391453 0.14090844 0.25391778 0.01544652]
Doc 2: It's Sunday, I need a break, so I'm planning to spend as little time as possible on the #A14.....
[0.02009892 0.02035123 0.11180288 0.82753472 0.02021225]
Doc 3: Awake but tired. I need to sleep but my brain has other ideas.....
[0.88244071 0.02977629 0.03000371 0.02910122 0.02867807]
Doc 4: RT @SewHQ: #Retro bears make perfect gifts and are great for beginners too! Get stitching with Octob...
[0.02938188 0.43047018 0.0288014 0.3177872 0.19355934]
Doc 5: It's hard to say whether packing lists are making life easier or just reinforcing how much still nee...
[0.01846263 0.1609002 0.28994117 0.25327571 0.2774203]
Doc 6: Making packing lists is my new hobby... #movinghouse...
[0.04097003 0.04119603 0.34048364 0.30224207 0.27510823]
Doc 7: At what point does keeping stuff for nostalgic reasons cross the line into plain old hoarding...? #m...
[0.90854551 0.02328548 0.02287758 0.02255801 0.02273342]
Doc 8: Currently in the finding-boxes-of-random-shit packing phase. I think I'm a closet hoarder.....
[0.04029868 0.28759878 0.29630566 0.04100147 0.33479541]
Doc 9: Can't be bothered to cook, take away on the way 🍕👍 #lazy...
[0.0401827 0.04078147 0.63485131 0.04042445 0.24376008]
Doc 10: RT @itventsnews: ITV releases promo video for the final series of Downton Abbey <http://t.co/mC8Ive72>...
[0.25668835 0.36485509 0.04017872 0.29817055 0.04010728]

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA

Tópico 1: depression (0.045), yong (0.017), treatments (0.017), thanks (0.014), best (0.014), overcome (0.013), hey (0.012), follow (0.012), video (0.009), good (0.009)
Tópico 2: like (0.022), love (0.015), people (0.013), just (0.010), depression (0.009), make (0.009), help (0.008), mental (0.008), look (0.007), need (0.007)
Tópico 3: im (0.050), dont (0.023), like (0.019), just (0.014), people (0.012), fuck (0.010), ive (0.009), know (0.008), love (0.008), trump (0.008)
Tópico 4: just (0.021), know (0.013), day (0.012), oh (0.011), want (0.010), amp (0.009), got (0.009), happy (0.008), dont (0.008), youre (0.008)
Tópico 5: thank (0.026), say (0.023), twitter (0.021), following (0.015), hello (0.014), good (0.014), way (0.011), girl (0.009), morning (0.009), anytime (0.009)

Forma prática

$$P(w | k) = \frac{n_{k,w} + \beta}{\sum_{w'} (n_{k,w'} + \beta)}$$

Significado dos termos

- $n_{k,w}$ → quantas vezes a palavra w foi atribuída ao tópico k
- β → suavização (evita zero)
- denominador → soma sobre todo o vocabulário

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA

1. Definições iniciais (antes dos dados)

Define **K tópicos** (fixo)

Para cada tópico k :

- gera uma distribuição de palavras ϕ_k sobre o vocabulário

2. Para cada documento d

Gera uma distribuição de tópicos:

- θ_d → mistura de tópicos do documento

3. Para cada palavra do documento

Para cada posição n :

Escolhe um tópico:

- $z_{d,n} \sim \theta_d$

Escolhe uma palavra:

- $w_{d,n} \sim \phi_{z_{d,n}}$

$$p(w|\alpha, \beta) = \prod_{k=1}^K p(\phi_k|\beta) \prod_{j=1}^M p(\theta_j|\alpha) \left(\prod_{i=1}^V p(z_{j,i}|\vec{\theta}_j) p(w_{i,j}|z_{j,i}, \phi_{z_{j,i}}) \right)$$

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA

O modelo assume que, ao gerar um documento, primeiro escolhe-se uma distribuição de tópicos. Em seguida, para cada palavra do documento, seleciona-se um tópico com base nessa distribuição e, a partir dele, uma palavra específica. Esse processo probabilístico permite estimar tanto os tópicos quanto sua presença em cada documento.

O LDA opera sobre representações vetoriais do tipo Bag of Words, nas quais a ordem das palavras é desconsiderada e apenas suas frequências são utilizadas. Essa simplificação permite capturar padrões estatísticos de coocorrência, fundamentais para a identificação dos tópicos.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - LDA

O principal parâmetro do LDA é o **número de tópicos** (K), que deve ser definido previamente. Esse valor influencia diretamente a granularidade dos temas identificados. Valores baixos tendem a gerar tópicos mais amplos, enquanto valores altos podem resultar em tópicos mais específicos ou redundantes. A escolha inadequada de K pode comprometer a interpretabilidade dos tópicos gerados.

O LDA é amplamente utilizado em tarefas exploratórias, como **identificação de temas em grandes coleções de textos, agrupamento de documentos por similaridade temática e apoio à análise qualitativa de conteúdo**.

Também pode ser utilizado como etapa de redução de dimensionalidade ao reduzir documentos de um espaço de palavras (alta dimensionalidade) para um espaço de tópicos de dimensão K .

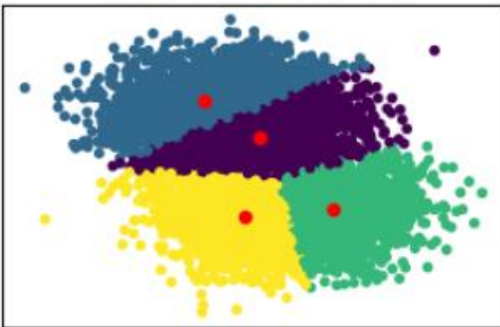
ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

K-Means

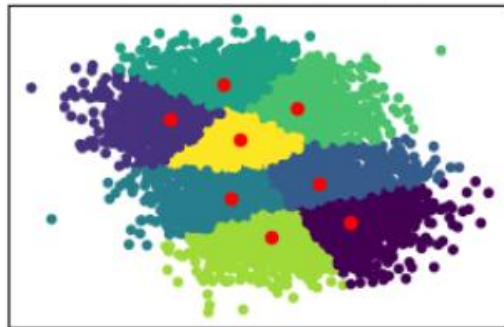
É um algoritmo de **clusterização** não supervisionada cujo objetivo é agrupar dados por **similaridade**. A ideia central é definir um centroide para cada grupo, calculado como a **média dos pontos**, e agrupar dados em torno dos centroides.

Assim, cada grupo é representado por um centroide e **cada ponto é associado ao cluster cujo centroide está mais próximo**. O algoritmo busca formar grupos com baixa variabilidade interna e alta separação entre si.

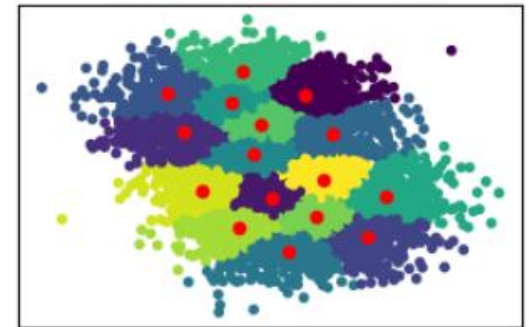
K-Means : 4 clusters



K-Means : 8 clusters



K-Means : 16 clusters



ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO — K-MEANS

Passo a Passo

- Primeiro define-se **k**, ou seja, o número de clusters desejado.
- Em seguida, o algoritmo inicializa os **centroides** (de forma aleatória, *init='random'*, ou usando **k-means++**, *init = 'k-means++'*).
- Depois, cada ponto é associado ao centroide mais próximo.
- Com os grupos formados, os centroides são recalculados como a **média dos pontos de cada cluster**.
- Esse processo de atribuição e recálculo se repete até que:
 - os centroides não mude mais; ou
 - que a mudança seja muito pequena (como *tol = 1e-4*); ou
 - até atingir um limite de iterações (como *max_iter = 10*).

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Função de Custo

Minimizar a soma das distâncias quadráticas até os centroides, implica reduzir o valor da função custo (J). Essa função mede o erro total do agrupamento, ou seja, o quanto os pontos estão distantes dos centros dos seus respectivos clusters. Quanto **menor o valor** de (J), **mais compactos** estão os grupos.

$\sum_{i=1}^k$ (somatório dos clusters)

Percorre todos os clusters definidos no problema. Para cada cluster, soma o erro interno dele. No final, agrega o erro de todos os grupos.

$\sum_{x \in C_i}$ (somatório dos pontos do cluster)

Percorre todos os pontos pertencentes ao cluster i . Para cada ponto, calcula o quanto ele está distante do centroide. Soma o erro de todos os pontos daquele cluster.

$\sum_{j=1}^d$ (somatório das dimensões)

Percorre cada atributo do ponto. Calcula a diferença em cada dimensão separadamente. Soma essas diferenças para formar a distância total do ponto ao centroide.

$$J = \sum_{i=1}^k \sum_{x \in C_i} \sum_{j=1}^d (x_j - \mu_{i,j})^2$$

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Função de Custo

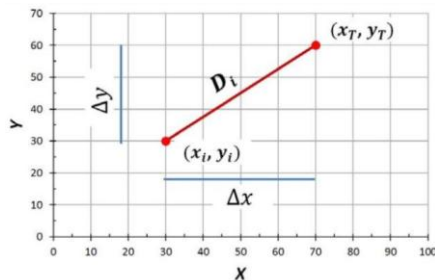
Dado um conjunto de pontos bidimensionais (1,1), (2,1), (5,5) e (6,5), aplique o algoritmo **K-Means** considerando $k = 2$. Realize a inicialização dos centroides, a atribuição dos pontos aos clusters e o recálculo dos centroides após a 1ª iteração.

1) Na **inicialização**, escolhemos dois centroides iniciais ($k = 2$). Suponha como centroides iniciais: $\mu_1 = (1,1)$ e $\mu_2 = (5,5)$

$$d(x, \mu_i) = \sqrt{\sum_{j=1}^d (x_j - \mu_{i,j})^2}$$

2) Na **associação**, cada ponto é comparado aos centroides e associado ao mais próximo. Assim, (1,1) e (2,1) ficam no cluster μ_1 ; enquanto (5,5) e (6,5) ficam no cluste μ_2 . Essa comparação de distância é, por padrão, a distância Euclidiana.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS



$$D = \sqrt{(x_T - x_i)^2 + (y_T - y_i)^2}$$

$$d(x, \mu_i) = \sqrt{\sum_{j=1}^d (x_j - \mu_{i,j})^2}$$

Assumindo os centróides:

- $\mu_1 = (1, 1)$
- $\mu_2 = (5, 5)$

Cálculos:

Ponto (2,1):

- $d(\mu_1) = \sqrt{((2-1)^2 + (1-1)^2)} = 1 \rightarrow \mu_1$
- $d(\mu_2) = \sqrt{((2-5)^2 + (1-5)^2)} = \sqrt{25} = 5$

Ponto (6,5):

- $d(\mu_1) = \sqrt{((6-1)^2 + (5-1)^2)} = \sqrt{41} \approx 6.4 \rightarrow \mu_2$
- $d(\mu_2) = \sqrt{((6-5)^2 + (5-5)^2)} = 1$

Com os centróides iniciais:

$$\mu_1 = (1, 1), \quad \mu_2 = (5, 5)$$

Associações:

- $(1, 1)$ e $(2, 1) \rightarrow \mu_1$
- $(5, 5)$ e $(6, 5) \rightarrow \mu_2$

Função de custo:

$$J = [(1-1)^2 + (1-1)^2] + [(2-1)^2 + (1-1)^2] \\ + [(5-5)^2 + (5-5)^2] + [(6-5)^2 + (5-5)^2]$$

$$J = 0 + 1 + 0 + 1$$

$$J = 2$$

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Função de Custo

Dado um conjunto de pontos bidimensionais $(1,1)$, $(2,1)$, $(5,5)$ e $(6,5)$, aplique o algoritmo **K-Means** considerando $k = 2$. Realize a inicialização dos centroides, a atribuição dos pontos aos clusters e o recálculo dos centroides após a 1ª iteração.

3) Depois disso, **recalculamos os centroides**. O novo centroide de cada cluster passa a ser a **média dos pontos pertencentes** a ele.

$$\mu_1 = \frac{(1,1) + (2,1)}{2}$$

$$\mu_1 = (1.5, 1)$$

$$\mu_2 = \frac{(5,5) + (6,5)}{2}$$

$$\mu_2 = (5.5, 5)$$

4) Depois, calculamos a **função custo J** , somando as **distâncias quadráticas de cada ponto ao centroide do seu cluster**. Quanto menor o valor de J , mais próximos os pontos estão dos centroides dos seus grupos.

$$\mu_1 = (1.5, 1), \mu_2 = (5.5, 5)$$

$$J = [(1 - 1.5)^2 + (1 - 1)^2] + [(2 - 1.5)^2 + (1 - 1)^2] + [(5 - 5.5)^2 + (5 - 5)^2] + [(6 - 5.5)^2 + (5 - 5)^2]$$

$$J = 0.25 + 0.25 + 0.25 + 0.25 = 1$$

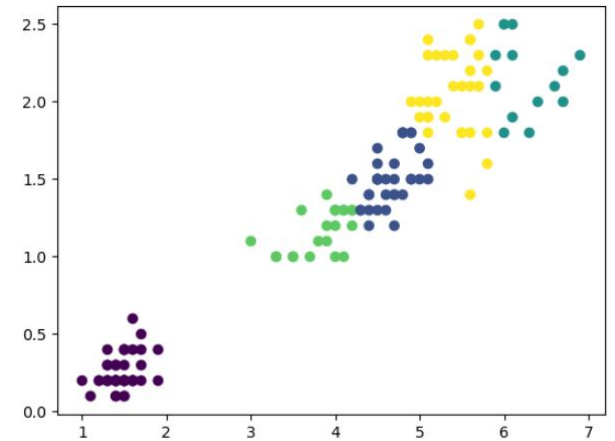
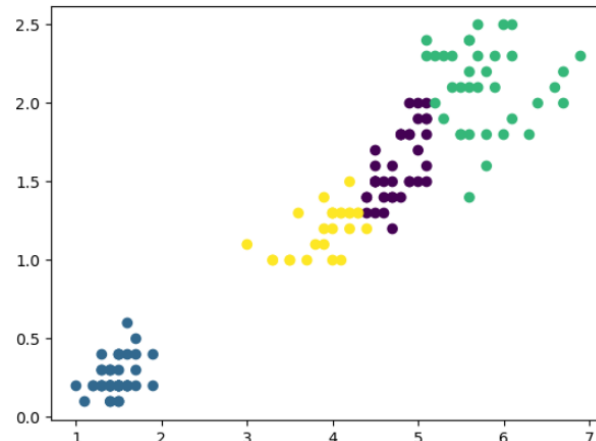
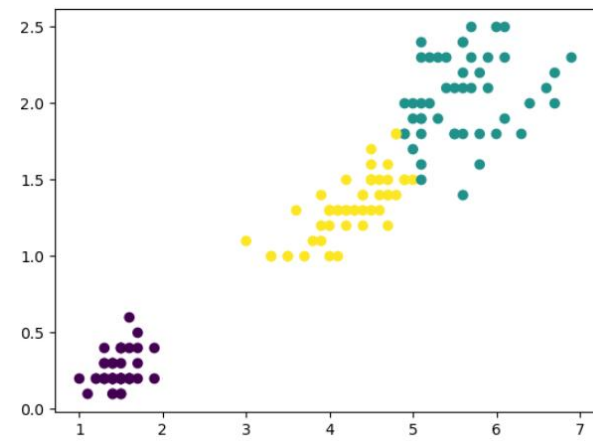
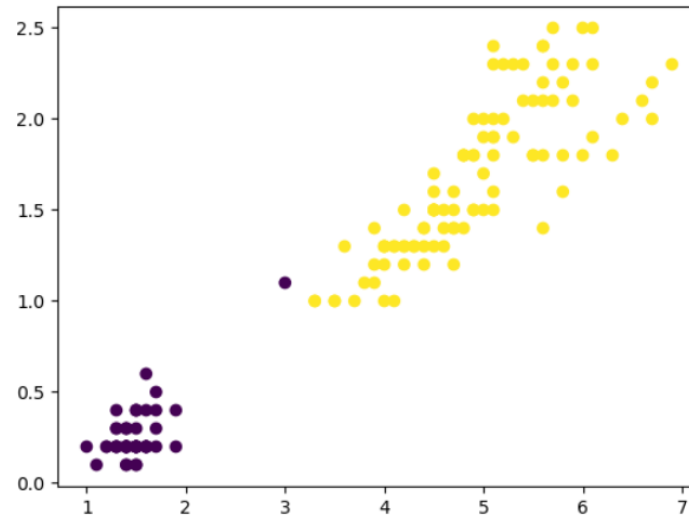
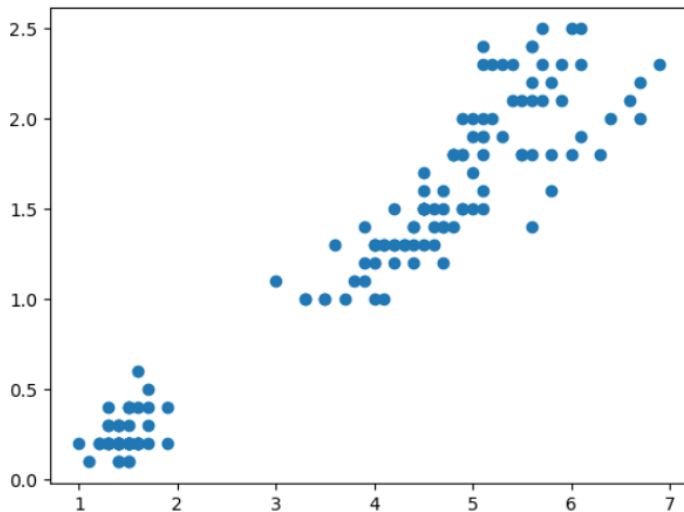
ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Escolha do Hiperparâmetro “K”

É um parâmetro crítico porque define **quantos grupos serão formados**. Esse valor não é aprendido automaticamente, portanto precisa ser definido previamente pelo analista. Ele impacta diretamente o resultado e, uma vez escolhido, **permanece fixo durante todo o processo de convergência**.

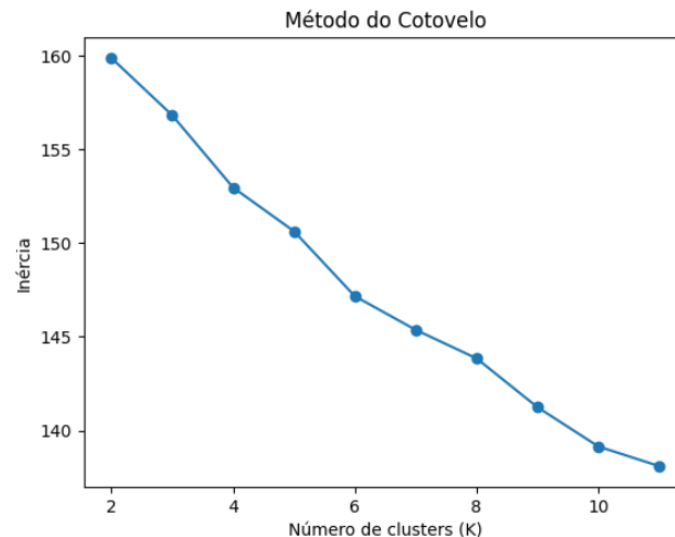
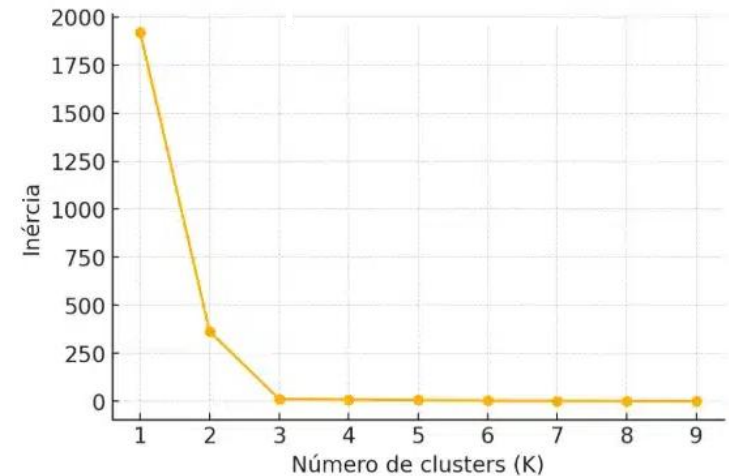
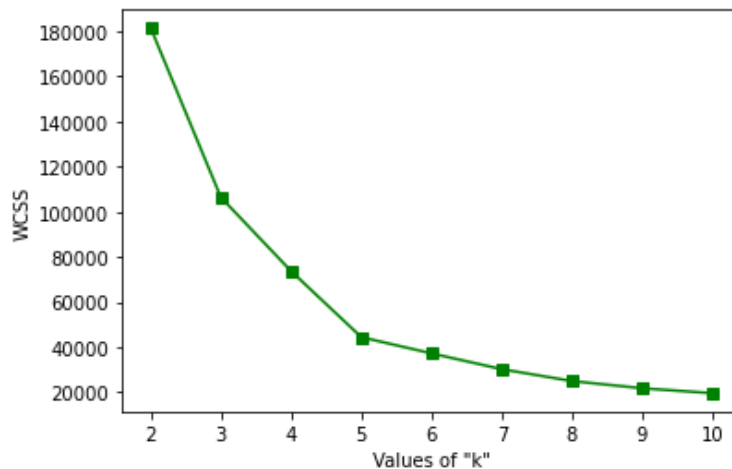
- A escolha **heurística** consiste em definir o valor de K com base no **conhecimento do problema e nos objetivos do estudo**. Não é uma escolha aleatória, mas sim uma decisão informada, por exemplo, quando já se sabe que existem certos **perfis naturais nos dados** ou quando há limitações práticas.
- Quando não existe uma preferência baseada no negócio, o **método do cotovelo** (elbow) consiste em executar o K-Means para valores de K e analisar como a função custo J se comporta. O ponto onde a redução de J começa a desacelerar indica um bom compromisso entre qualidade e simplicidade, sugerindo um valor adequado de k .

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS



ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Método do Cotovelo (Elbow)



ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO — K-MEANS

Inicialização dos Centroides

A escolha inicial dos centroides é crítica, pois influencia diretamente o resultado final do algoritmo. No K-Means padrão, os centroides são definidos de **forma aleatória**, o que pode levar a diferentes soluções em execuções distintas e até a resultados ruins (mínimos locais).

- Para reduzir esse problema, utiliza-se o método **K-Means++**, que inicializa os centroides de forma mais estratégica, **escolhendo pontos mais distantes entre si**. Isso melhora a qualidade dos clusters e acelera a convergência.
- Na prática, implementações como o scikit-learn já utilizam **K-Means++ como padrão** (`init="k-means++"`), o que melhora a escolha inicial dos centroides.
- Além disso, é possível configurar o parâmetro **n_init**, que define quantas vezes o algoritmo será executado com diferentes inicializações. Ao final, é selecionada a solução com menor valor da função custo J .

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO — K-MEANS

Boas Práticas com K-Means

- Escalonar os dados, pois o K-means usa distância.
- Escolher k com método do cotovelo, evitando decisões arbitrárias.
- Inicializar com k-means++, para evitar soluções ruins.
- Executar múltiplas vezes com n_init, para reduzir o risco de mínimo local.
- Avaliar o resultado, não confiando apenas na inércia.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Índice de Rand (Rand Index – RI)

Mede a similaridade entre duas formas de agrupar os dados, comparando os clusters **obtidos pelo algoritmo** com uma **referência de rótulos reais**.

A avaliação é feita com base em **pares de pontos**, verificando se dois elementos são agrupados de forma consistente nos dois agrupamentos (clusters versus rótulos). Quanto maior a concordância entre essas decisões, maior o valor do índice (varia de **0 a 1**, sendo que valores mais próximos de 1 indicam maior similaridade).

$$RI = \frac{a + b}{a + b + c + d}$$

$$RI = \frac{a + b}{\binom{n}{2}}$$

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Índice de Rand (Rand Index – RI)

Índice Rand:

$$RI = \frac{a + b}{a + b + c + d}$$

Equivalente:

$$RI = \frac{a + b}{\binom{n}{2}} = \frac{a + b}{\frac{n(n-1)}{2}}$$

a = pares na mesma classe e no mesmo cluster (acerto)

b = pares em classes diferentes e em clusters diferentes (acerto)

c = pares na mesma classe e em clusters diferentes (erro)

d = pares em classes diferentes e no mesmo cluster (erro)

$(a + b)$ = total de acertos

$(c + d)$ = total de erros

○ Índice Rand pode apresentar valores elevados mesmo para agrupamentos aleatórios, pois a grande quantidade de pares corretamente separados (**b**) tende a inflar o resultado. Em datasets reais, a maioria dos pares vai estar mesmo em **classes diferentes**.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO – K-MEANS

Índice Rand Ajustado (Adjusted Rand Index – ARI)

Para corrigir essa limitação, utiliza-se o ARI, que considera o “valor esperado ao acaso”. Isso corrige a principal limitação do Índice Rand ao considerar o efeito do acaso na formação dos clusters.

Seus valores variam de -1 a 1.

O valor esperado $E[RI]$ é calculado teoricamente, assumindo associações aleatórias entre clusters e classes, mantendo fixos os tamanhos dos grupos.

$$ARI = \frac{RI - E[RI]}{1 - E[RI]}$$

$$E[RI] = \frac{\sum_i \binom{a_i}{2} \cdot \sum_j \binom{b_j}{2}}{\binom{n}{2}^2}$$

Onde:

a_i = tamanho do cluster i

b_j = tamanho da classe real j

n = número total de pontos

$\binom{x}{2}$ = número de pares possíveis em um grupo de tamanho x

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO — K-MEANS

Coeficiente de Silhueta (Silhouette Score)

Mede o quão bem cada ponto está associado ao seu próprio cluster em comparação com outros clusters.

Ele ajuda a avaliar simultaneamente a **coesão** (proximidade dentro do cluster) e a **separação** (distância para outros clusters).

O coeficiente de silhueta é obtido **individualmente para cada ponto**, e o valor final do modelo corresponde à **média desses valores** em todo o conjunto de dados.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO — K-MEANS

Coeficiente de Silhueta (Silhouette Score)

Mede o quão bem cada ponto está **associado** ao seu próprio cluster em comparação com outros clusters.

Para **cada ponto** do conjunto de dados, calcula-se a **distância média** em relação aos **pontos do mesmo cluster**, representando a **coesão**.

Em seguida, calcula-se a **menor distância média** em relação aos **pontos de todos os outros clusters**, representando a **separação**.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Onde:

$a(i)$ = distância média do ponto i para os pontos do mesmo cluster

$b(i)$ = menor distância média do ponto i para pontos de outro cluster



ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO — K-MEANS

Quando usar?

Dados numéricos vetorizados (ex: TF-IDF)

Estrutura aproximadamente esférica

Segmentação exploratória

Desvantagens

Sensível a outliers

Sensível à inicialização

Assume clusters convexos e similares



V - PLN MODERNO

REPRESENTAÇÃO SEMÂNTICA

Prof. Luis Claudio

M.Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)

PLN MODERNO

EMBEDDINGS

Em engenharia de atributos nós vimos:

- **Segmentação** (definir unidades linguísticas de análise)
 - Tokenização
- **Normalização textual** (reduzir variações formais não informativas)
 - Remoção de Stop Words, Stemização e Lematização.
- **Extração de informação** (estruturar elementos linguísticos relevantes)
- **Representação vetorial do texto** (converter texto em espaço numérico modelável)
 - One-Hot → representação discreta (presença)
 - BoW → baseadas em frequência bruta
 - TF-IDF → baseadas em frequência ponderada
 - PPMI → baseada em coocorrência
 - **Embeddings** → baseados em contexto/semântica (Word2Vec e SBERT)

ONE HOT

Build ideas with logic and code

One Hot
Encoding
→

Build	1	0	0	0	0	0
ideas	0	1	0	0	0	0
with	0	0	1	0	0	0
logic	0	0	0	1	0	0
and	0	0	0	0	1	0
code	0	0	0	0	0	1

BOW

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]  
  
[('bebe', 5),  
 ('come', 5),  
 ('gato', 4),  
 ('cachorro', 4),  
 ('agua', 3),  
 ('carne', 3),  
 ('leite', 2),  
 ('peixe', 2),  
 ('homem', 2)]
```

bow

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
D1	1	1	0	0	0	1	0	0	0
D2	1	1	1	0	0	0	0	0	0
D3	0	1	0	0	0	1	0	1	0
D4	0	1	1	0	0	0	0	1	0
D5	0	0	0	0	1	1	0	0	1
D6	0	0	1	0	1	0	0	0	1
D7	0	0	0	1	1	1	0	0	0
D8	0	0	1	1	1	0	0	0	0
D9	1	1	0	0	0	0	1	0	0
D10	0	0	0	1	1	0	1	0	0

TF-IDF

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]
```

$$w_{i,j} = tf_{i,j} \times \log \left(\frac{N}{df_i} \right)$$

onde:

$tf_{i,j}$ = frequência do termo i no documento j

N = número total de documentos no corpus

df_i = número de documentos que contêm o termo i

TF-IDF(t, d) mede o quão relevante o termo t é para o documento d dentro do corpus.

- TF (Term Frequency) → quantas vezes o termo aparece no documento
- IDF (Inverse Document Frequency) → quão raro esse termo é no corpus

👉 TF-IDF = importância local × raridade global

Lendo a tabela (exemplo D1):

- `agua = 0.641771` → aparece em D1 e não é tão comum no corpus → relevante
- `bebe = 0.512414` → relevante, mas menos que "água"
- `gato = 0.570581` → também relevante
- demais = 0 → não aparecem em D1

tfidf

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
D1	0.641771	0.512414	0.000000	0.000000	0.000000	0.570581	0.000000	0.000000	0.000000
D2	0.641771	0.512414	0.570581	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
D3	0.000000	0.482845	0.000000	0.000000	0.000000	0.537655	0.000000	0.691222	0.000000
D4	0.000000	0.482845	0.537655	0.000000	0.000000	0.000000	0.000000	0.691222	0.000000
D5	0.000000	0.000000	0.000000	0.000000	0.482845	0.537655	0.000000	0.000000	0.691222
D6	0.000000	0.000000	0.537655	0.000000	0.482845	0.000000	0.000000	0.000000	0.691222
D7	0.000000	0.000000	0.000000	0.641771	0.512414	0.570581	0.000000	0.000000	0.000000
D8	0.000000	0.000000	0.570581	0.641771	0.512414	0.000000	0.000000	0.000000	0.000000
D9	0.582818	0.465343	0.000000	0.000000	0.000000	0.000000	0.666168	0.000000	0.000000
D10	0.000000	0.000000	0.000000	0.582818	0.465343	0.000000	0.666168	0.000000	0.000000

PMI

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]
```

bigrams_list

```
[('gato', 'bebe'),  
 ('bebe', 'agua'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'agua'),  
 ('gato', 'bebe'),  
 ('bebe', 'leite'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'leite'),  
 ('gato', 'come'),  
 ('come', 'peixe'),  
 ('cachorro', 'come'),  
 ('come', 'peixe'),  
 ('gato', 'come'),  
 ('come', 'carne'),  
 ('cachorro', 'come'),  
 ('come', 'carne'),  
 ('homem', 'bebe'),  
 ('bebe', 'agua'),  
 ('homem', 'come'),  
 ('come', 'carne')]
```

bigram_freq

```
Counter({'gato', 'bebe'): 2,  
        ('bebe', 'agua'): 3,  
        ('cachorro', 'bebe'): 2,  
        ('bebe', 'leite'): 2,  
        ('gato', 'come'): 2,  
        ('come', 'peixe'): 2,  
        ('cachorro', 'come'): 2,  
        ('come', 'carne'): 3,  
        ('homem', 'bebe'): 1,  
        ('homem', 'come'): 1})
```

Calcular total de coocorrências.

N = matrix.values.sum()

N

np.int64(20)

p_wc = matrix / N

p_wc

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
bebe	0.15	0.00	0.0	0.00	0.00	0.0	0.0	0.1	0.0
cachorro	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
carne	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
come	0.00	0.00	0.0	0.15	0.00	0.0	0.0	0.0	0.1
gato	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
homem	0.00	0.05	0.0	0.00	0.05	0.0	0.0	0.0	0.0
leite	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
peixe	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0

PMI

```
corpus = [  
    "gato bebe agua",  
    "cachorro bebe agua",  
    "gato bebe leite",  
    "cachorro bebe leite",  
    "gato come peixe",  
    "cachorro come peixe",  
    "gato come carne",  
    "cachorro come carne",  
    "homem bebe agua",  
    "homem come carne",  
]
```

bigrams_list

```
[('gato', 'bebe'),  
 ('bebe', 'agua'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'agua'),  
 ('gato', 'bebe'),  
 ('bebe', 'leite'),  
 ('cachorro', 'bebe'),  
 ('bebe', 'leite'),  
 ('gato', 'come'),  
 ('come', 'peixe'),  
 ('cachorro', 'come'),  
 ('come', 'peixe'),  
 ('gato', 'come'),  
 ('come', 'carne'),  
 ('cachorro', 'come'),  
 ('come', 'carne'),  
 ('homem', 'bebe'),  
 ('bebe', 'agua'),  
 ('homem', 'come'),  
 ('come', 'carne')]
```

bigram_freq

```
Counter({'gato', 'bebe'): 2,  
        ('bebe', 'agua'): 3,  
        ('cachorro', 'bebe'): 2,  
        ('bebe', 'leite'): 2,  
        ('gato', 'come'): 2,  
        ('come', 'peixe'): 2,  
        ('cachorro', 'come'): 2,  
        ('come', 'carne'): 3,  
        ('homem', 'bebe'): 1,  
        ('homem', 'come'): 1})
```

Calcular total de coocorrências.

N = matrix.values.sum()

N

np.int64(20)

p_w

	0
agua	0.00
bebe	0.25
cachorro	0.20
carne	0.00
come	0.25
gato	0.20
homem	0.10
leite	0.00
peixe	0.00

p_c

	0
agua	0.15
bebe	0.25
cachorro	0.00
carne	0.15
come	0.25
gato	0.00
homem	0.00
leite	0.10
peixe	0.10

p_wc = matrix / N

p_wc

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
bebe	0.15	0.00	0.0	0.00	0.00	0.0	0.0	0.1	0.0
cachorro	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
carne	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
come	0.00	0.00	0.0	0.15	0.00	0.0	0.0	0.0	0.1
gato	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
homem	0.00	0.05	0.0	0.00	0.05	0.0	0.0	0.0	0.0
leite	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
peixe	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0

100

1

```
[('gato', 'bebe'),
 ('bebe', 'agua'),
 ('cachorro', 'bebe'),
 ('bebe', 'agua'),
 ('gato', 'bebe'),
 ('bebe', 'leite'),
 ('cachorro', 'bebe'),
 ('bebe', 'leite'),
 ('gato', 'come'),
 ('come', 'peixe'),
 ('cachorro', 'come'),
 ('come', 'peixe'),
 ('gato', 'come'),
 ('come', 'carne'),
 ('cachorro', 'come'),
 ('come', 'carne'),
 ('homem', 'bebe'),
 ('bebe', 'agua'),
 ('homem', 'come'),
 ('come', 'carne')]
```

$$PMI(w,c) = \log(P(w,c) / (P(w) \times P(c)))$$

$P(w,c)$ = probabilidade conjunta do par palavra-contexto

$P(c)$ = probabilidade marginal do contexto

```
Counter({'gato', 'bebe': 2,
        ('bebe', 'agua'): 3,
        ('cachorro', 'bebe'): 2,
        ('bebe', 'leite'): 2,
        ('gato', 'come'): 2,
        ('come', 'peixe'): 2,
        ('cachorro', 'come'): 2,
        ('come', 'carne'): 3,
        ('homem', 'bebe'): 1,
        ('homem', 'come'): 1})
```

[illegible]

PLN MODERNO

EMBEDDINGS

1) Ambiguidade (Polissemia)

Problema: uma mesma palavra pode ter significados diferentes.

- Exemplo:
- “Ele sentou no banco.”
- “Ele foi ao banco.”

Em **BoW** e **TF-IDF**, “banco” é apenas um termo do vocabulário.

O modelo não entende sentidos (só conta **quantas vezes** cada palavra aparece).

PLN MODERNO

EMBEDDINGS

2) Variabilidade (Sinonímia)

Problema: um mesmo significado pode ter palavras diferentes.

- Exemplo:

Doc A: “O carro é rápido.”

Doc B: “O automóvel é rápido.”

Doc C: “ O veículo é veloz.”

Tanto para **BoW** quanto para o **TF-IDF**: “carro” $\neq$ “automóvel” $\neq$ “veículo”.

Ou seja, o modelo não capta o significado comum entre os textos. Para ele, ‘carro’, ‘automóvel’ e ‘veículo’ são termos distintos, cada um aparecendo poucas vezes. Como não há sobreposição de palavras, ele não identifica que os documentos tratam do mesmo tema (**automobilismo**), mesmo sendo semanticamente equivalentes.

PLN MODERNO

EMBEDDINGS

3) Dependência de Contexto (Relações Semânticas Latentes)

Problema: textos podem falar do mesmo tema com outras palavras.

- Exemplo:

Doc A: “Ele comprou ações na bolsa de valores.”

Doc B: “O mercado financeiro caiu devido à crise.”

Doc C: “O urso tomou conta do mercado.”

Os textos falam sobre finanças. Mas não compartilham palavras iguais.

Ou seja, o modelo não captura relações semânticas implícitas. Mesmo tratando do mesmo tema (finanças), os textos usam vocabulários diferentes e, por isso, têm pouca ou nenhuma sobreposição de termos. Para BoW e TF-IDF, ‘ações’, ‘bolsa’ e ‘mercado financeiro’ são apenas palavras distintas, sem conexão entre si.

Muito menos o termo ‘urso’ (no sentido de mercado em queda), pois esse tipo de relação só é capturado por modelos semânticos mais avançados.

PLN MODERNO

WORD2VEC

Word2Vec - Motivação

As representações clássicas de texto, como One Hot, Bag-of-Words e TF-IDF, tratam palavras como **unidades independentes** e baseiam-se apenas na **frequência** ou **presença de termos**.

Isso faz com que o modelo não capture relações semânticas: palavras diferentes com o mesmo significado (sinonímia) são tratadas como completamente distintas, e palavras com múltiplos sentidos (ambiguidade) são representadas sempre da mesma forma.

A similaridade entre textos depende da sobreposição exata de palavras, ignorando contexto e significado. Diante dessas limitações, surge a necessidade de uma representação que capture relações semânticas e contextuais entre palavras, permitindo que o modelo entenda similaridade de significado e não apenas coincidência lexical.

PLN MODERNO

WORD2VEC

Word2Vec - Ideia Central

O Word2Vec parte do princípio de que o significado de uma palavra pode ser inferido a partir do **contexto** em que ela aparece.

Em vez de contar palavras, o modelo aprende **representações numéricas** (vetores) de forma que palavras que ocorrem em contextos semelhantes tenham **vetores próximos no espaço vetorial**.

Assim, a **linguagem passa a ser representada geometricamente: similaridade semântica é convertida em proximidade entre vetores**. Esse aprendizado é feito de forma automática a partir de grandes volumes de texto, capturando padrões de uso e relações implícitas entre palavras.

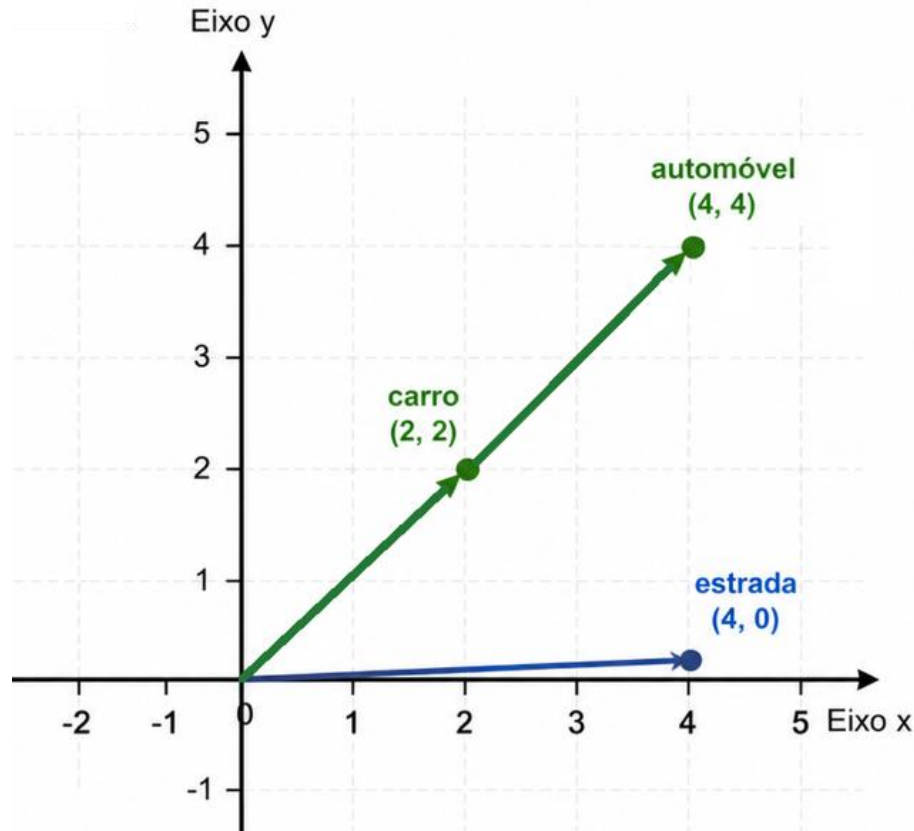
PLN MODERNO

WORD2VEC

Cosine similarity :

- 1 → mesma direção (máxima similaridade)
- 0 → ortogonais (sem relação)
- -1 → opostos

Similaridade no Word2Vec



1) carro vs. automóvel

- Vetores na mesma direção (paralelos)
- Ângulo $\approx 0^\circ$
- Similaridade de cosseno ≈ 1 (muito alta)

2) carro vs. estrada

- Vetores com direções diferentes
- Ângulo $\approx 45^\circ$
- Similaridade de cosseno $\approx 0,71$ (mais baixa)

PLN MODERNO

WORD2VEC

Word2Vec - Como Funciona

O Word2Vec utiliza uma **rede neural rasa** (*shallow*) para aprender representações vetoriais (embeddings) das palavras a partir de tarefas de predição baseadas no contexto.

Existem duas abordagens principais: no modelo **CBOW** (Continuous Bag of Words), o objetivo é prever uma palavra a partir do seu contexto; já no **Skip-gram**, o objetivo é prever o contexto a partir de uma palavra.

Em ambos os casos, define-se uma *janela de contexto* que determina quais palavras vizinhas serão consideradas. Durante o treinamento, os **pesos da rede são ajustados** para maximizar a probabilidade das predições, e os vetores aprendidos na camada intermediária passam a representar semanticamente as palavras.

PLN MODERNO

WORD2VEC

Word2Vec - Rede Neural

O Word2Vec utiliza uma **rede neural rasa** (*shallow*) com três camadas: entrada, projeção e saída. A entrada é um vetor **one-hot** da palavra (ou contexto), que é projetado para um espaço de menor dimensão (**vector_size**) por meio de uma matriz de pesos.

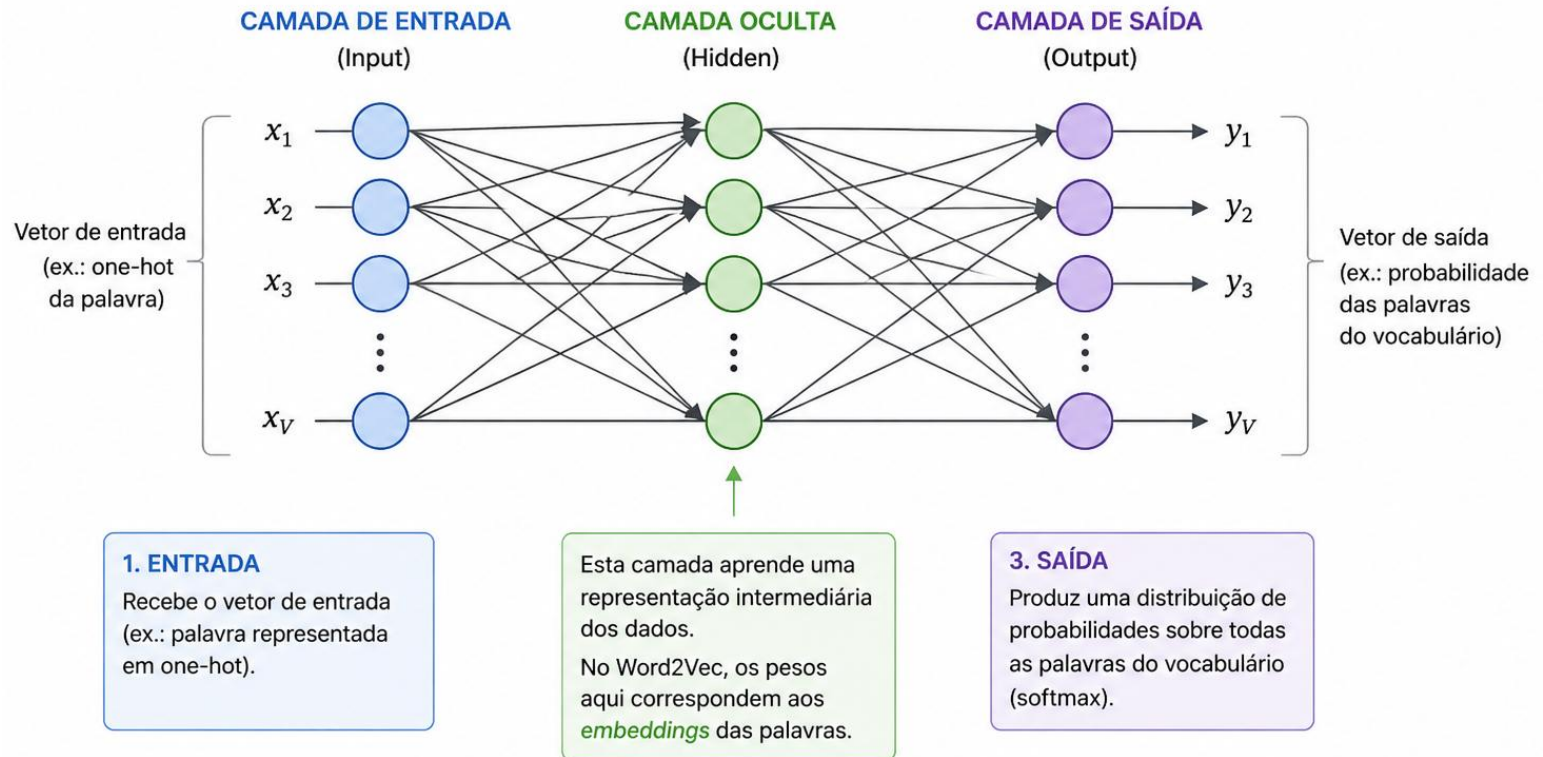
Essa camada intermediária corresponde diretamente ao embedding aprendido. A saída tenta prever palavras (contexto ou palavra central), geralmente usando uma função como *softmax*.

O treinamento ajusta os pesos para maximizar a probabilidade dessas previsões, com foco em fazer com **palavras com contextos semelhantes tenham representações vetoriais próximas**.

PLN MODERNO

WORD2VEC

Rede Neural Rasa (Shallow)



PLN MODERNO

WORD2VEC

Vocabulário:

gato → 0
bebe → 1
leite → 2

Embedding = 2

Matriz W1 (embeddings)

W1 (3×2)

gato → [0.2, 0.8]
bebe → [0.5, 0.1]
leite → [0.9, 0.4]

Matriz W2 (saída)

W2 (2×3)

gato bebe leite
[0.2] → [0.3, 0.7, 0.5]
[0.8] → [0.6, 0.2, 0.4]

Par de treino (Skip-gram)

gato → bebe

1. Entrada (one-hot)

gato → [1, 0, 0]

2. Camada oculta (embedding)

$[1, 0, 0] \times W1 = [0.2, 0.8]$

Esse vetor é o embedding de “gato”

3. Projeção para saída (scores)

$[0.2, 0.8] \times W2$

gato = $0.2 \times 0.3 + 0.8 \times 0.6 = 0.54$

bebe = $0.2 \times 0.7 + 0.8 \times 0.2 = 0.30$

leite = $0.2 \times 0.5 + 0.8 \times 0.4 = 0.42$

Resultado (scores): [0.54, 0.30, 0.42]

4. Softmax (probabilidades)

$[0.54, 0.30, 0.42] \rightarrow [0.40, 0.28, 0.32]$

Soma = 1

5. Saída esperada

bebe → [0, 1, 0]

6. Erro

modelo deu 0.28 para “bebe”

deveria dar 1

→ existe erro

7. Atualização

W1 e W2 são ajustadas

Exemplo após ajuste:

gato → [0.3, 0.7]

bebe → [0.4, 0.2]

“gato” fica mais próximo de “bebe”

PLN MODERNO

WORD2VEC

vector_size → dimensão do embedding (tamanho do vetor de cada palavra)
window → tamanho da janela de contexto (quantas palavras vizinhas considerar)
min_count → frequência mínima para incluir a palavra no vocabulário
sg → define o modelo: 0 = CBOW, 1 = Skip-gram
epochs → número de passadas completas pelo corpus
alpha → taxa de aprendizado inicial
min_alpha → taxa de aprendizado final (decai ao longo do treino)
negative → número de amostras negativas (negative sampling)
hs → ativa hierarchical softmax (1) ou desativa (0)
sample → reduz impacto de palavras muito frequentes (downsampling)
workers → número de threads usadas no treinamento
batch_words → quantidade de palavras processadas por bloco
max_vocab_size → limite máximo do vocabulário durante construção
max_final_vocab → tamanho final do vocabulário após corte
sorted_vocab → ordena vocabulário por frequência
compute_loss → calcula o erro durante o treinamento

PLN MODERNO

WORD2VEC

Word2Vec - CBOW vs Skip-gram

O Word2Vec pode ser treinado de **duas formas**.

Na prática:

- No **CBOW** (*Continuous Bag of Words*), o modelo recebe as palavras do contexto e tenta prever a palavra central. É mais rápido e funciona melhor com grandes volumes de dados mais “limpos”.
- Já no **Skip-gram**, o modelo recebe uma palavra central e tenta prever as palavras ao seu redor (o contexto). É mais lento, mas costuma capturar melhor relações semânticas, especialmente para palavras raras.

Ambos aprendem embeddings, mas com perspectivas “inversas” de predição.

PLN MODERNO

WORD2VEC

Frase = ["gato", "bebe", "leite", "na", "casa"]

window = 2

CBOW (contextos → palavra)

gato

([bebe, leite] → gato)

bebe

([gato, leite, na] → bebe)

leite

([gato, bebe, na, casa] → leite)

na

([bebe, leite, casa] → na)

casa

([leite, na] → casa)

CBOW → tecnicamente não é par simples

→ (contexto, palavra_central)

→ contexto = conjunto/lista

PLN MODERNO

WORD2VEC

Frase = ["gato", "bebe", "leite", "na", "casa"]

window = 2

Skip-gram (palavra → contextos)

gato

(gato → bebe)
(gato → leite)

bebe

(bebe → gato)
(bebe → leite)
(bebe → na)

leite

(leite → gato)
(leite → bebe)
(leite → na)
(leite → casa)

na

(na → bebe)
(na → leite)
(na → casa)

Skip-gram → pares (input, output)

→ (*palavra_central*, *palavra_contexto*)

PLN MODERNO

WORD2VEC

Word2Vec - Resultado

Ao final do treinamento, cada palavra do vocabulário é representada por um **vetor denso de baixa dimensionalidade**, que captura relações semânticas e padrões de uso observados no corpus. Esses vetores permitem calcular similaridade entre palavras por meio de medidas como a **similaridade de cosseno**, refletindo proximidade de significado.

Além disso, os embeddings podem capturar relações estruturais na linguagem, permitindo **operações vetoriais simples que refletem analogias semânticas**. Na prática, esses vetores passam a ser utilizados como entrada para modelos de aprendizado de máquina ou para tarefas como clustering e busca por similaridade.

PLN MODERNO

WORD2VEC

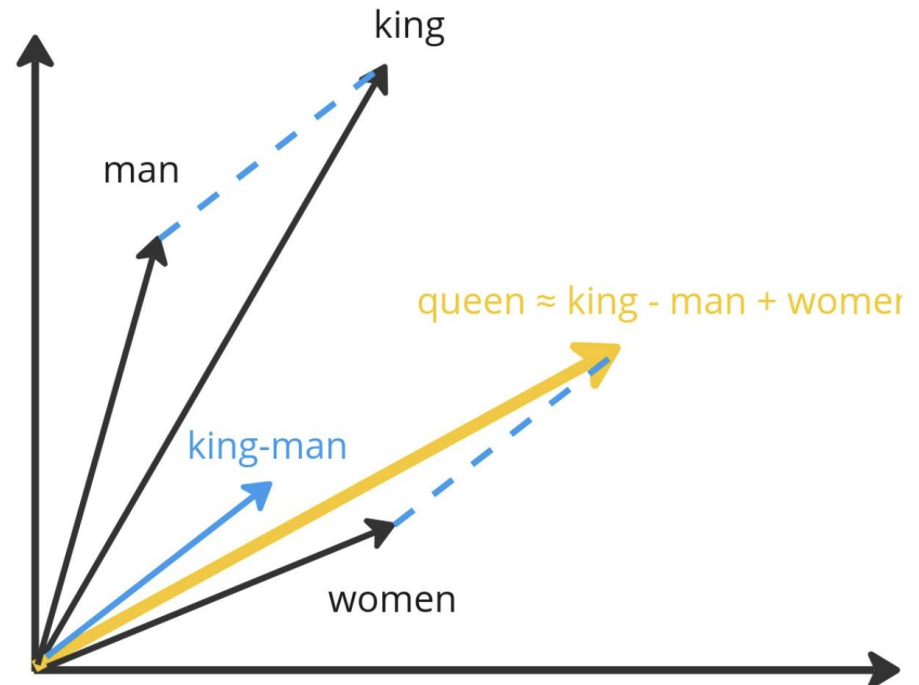
A imagem representa uma analogia clássica em Word2Vec:

$$\text{queen} \approx \text{king} - \text{man} + \text{woman}$$

A ideia é que o vetor “**king - man**” remove de “king” a dimensão associada a “homem”, preservando uma relação mais abstrata de “realeza” (seta azul).

Ao somar o vetor **woman**, o resultado (amarelo) se desloca para uma região próxima de **queen**. Assim, *embeddings* capturam relações semânticas como deslocamentos no espaço vetorial.

O ponto central é: relações de significado aparecem como **direções vetoriais** aprendidas a partir dos contextos em que as palavras ocorrem.



PLN MODERNO

WORD2VEC

Word2Vec - Limitações

Apesar de capturar relações semânticas, o Word2Vec atribui **um único vetor por palavra**, independentemente do contexto em que ela aparece. Ou seja, ele não produz diretamente representações para sentenças ou documentos, exigindo estratégias adicionais de agregação.

- O Word2Vec **não consegue distinguir diferentes sentidos de uma mesma palavra** (polissemia/ambiguidade), pois cada palavra tem apenas um vetor.
- O modelo não considera a **ordem das palavras** nem a **estrutura sintática completa**, limitando sua capacidade de representar nuances mais complexas da linguagem.

PLN MODERNO

WORD2VEC

SBERT - Motivação

○ Word2Vec gera embeddings no nível de palavras, atribuindo **um único vetor fixo para cada**. O contexto é considerado durante o treinamento para aprender os embeddings, mas o resultado final é um **vetor único por palavra** (uma “média” dos diferentes contextos em que ela apareceu).

○ SBERT (***Sentence - Bidirectional Encoder Representations from Transformers***) é um modelo baseado em Transformers que gera embeddings no nível de sentenças, incorporando o contexto das palavras na representação do texto. Diferentemente de abordagens no nível de palavras, como o Word2Vec, o SBERT produz um **único vetor para cada sentença ou documento**, preservando melhor o significado global.

Esses embeddings podem ser comparados diretamente por medidas de similaridade, o que torna o modelo adequado para tarefas como busca semântica, clustering e recomendação de textos.

PLN MODERNO

WORD2VEC

SBERT - Como Funciona

O SBERT utiliza uma arquitetura baseada em BERT combinada com estruturas siamese ou triplet, nas quais duas ou três sentenças são processadas em paralelo. Durante o treinamento, o modelo aprende a aproximar vetores de sentenças semanticamente semelhantes e afastar sentenças diferentes.

Após a passagem pelo BERT, aplica-se uma operação de pooling (como média dos tokens ou uso do token CLS) para gerar um único vetor representativo da sentença. Esse vetor final é otimizado diretamente para tarefas de similaridade semântica.

PLN MODERNO

WORD2VEC

SBERT - Resultado

O modelo gera um vetor denso para cada sentença ou documento, permitindo comparar textos diretamente por similaridade de cosseno. Diferentemente do Word2Vec, que produz vetores no nível de palavras e exige agregações que podem perder informação, o SBERT representa o significado da sentença como um todo, preservando melhor o contexto.

Na prática, isso permite aplicações como busca semântica, clustering de textos, deduplicação e sistemas de recomendação baseados em significado, com maior qualidade semântica.

PLN MODERNO

WORD2VEC

SBERT - Vantagens e Limitações

O SBERT captura contexto e significado de forma muito mais precisa do que Word2Vec ou TF-IDF, sendo adequado para comparação direta entre textos.

Além disso, permite escalabilidade em tarefas de similaridade, pois os embeddings podem ser pré-computados.

Por outro lado, o modelo é mais pesado computacionalmente, exige maior custo de processamento e memória, e **depende de modelos pré-treinados robustos**. Apesar de contextual, ainda pode apresentar limitações em **domínios muito específicos sem ajuste fino**.